

Upper Bounds for the Number of Possible Chess Positions

BACHELOR'S THESIS

Date of submission: 09/29/2023

at the Faculty of Mathematics, Natural Sciences, and
Technology of the University of Augsburg

Name:	Christian Büchs
Matriculation number:	1645356
Degree program:	Mathematics for Economics
First reviewer:	Prof. Dr. Dirk Hachenberger
Second reviewer:	Prof. Dr. André Uschmajew

Table of Contents

1	Introduction	2
1.1	Content of the Thesis	2
1.2	What is a Chess Position?	3
2	Shannon's Estimate	5
3	Chinchalkar's Solution	7
3.1	Calculating the Piece Combinations	7
3.2	Placing the Piece Combinations	15
4	Tromp's Solution	22
4.1	Fixed Pieces and Multipliers	22
4.2	Implementing the Piece Combinations	26
4.3	Calculating the Pawn Distribution	26
5	My Extension	40
6	Estimation via a Confidence Interval	48
6.1	Implementation of the Sample	48
6.2	Mathematical Background	51
6.3	Sample Results	52
7	Conclusion	54
8	References	55
9	Appendix	56

1 Introduction

1.1 Content of the Thesis

Over the past decades, the internet and the existence of computers have revolutionized our lives at a pace never seen before by any other invention. Computers relieve us of navigation using a map via Google Maps, they render letters obsolete through apps like WhatsApp, and they are used in almost all areas of life to calculate and simulate models. It is therefore not surprising that Claude Shannon as early as 1950, in his work "XXII. Programming a Computer for Playing Chess" [4], posed the question of whether it was possible to program a computer to play chess perfectly or at least on the level of a good chess player.

In that work, Shannon used the two numbers 10^{120} and 10^{43} as approximations for the number of possible chess games and the number of possible chess positions, in order to argue that there cannot be a perfectly-playing computer that makes a move in a reasonable time. From this proof, the two questions arose: "What is the smallest upper bound so far found for the number of possible chess positions?" and "What is a good estimate for the number of possible chess positions?"

In this Bachelor's thesis, we deal with these two questions. We will answer the first question by examining the various attempts of Chinchalkar, Tromp, and myself to find an upper bound that is as small as possible, and proving the respective approaches. For the second question, we will initially look at Shannon's estimate and, in conclusion, study Tromp's calculation of a confidence interval.

This thesis assumes that the rules of chess are known. A chess guide can be found at <https://www.brettspielnetz.de/spielregeln/schach.php>. Furthermore, for Chapter 6, knowledge from the course "Stochastics 1" is required.

1.2 What is a Chess Position?

Before we can address upper bounds and confidence intervals, we must define what a chess position actually is. A chess position is supposed to describe a game situation. In chess, two game situations can differ in the following aspects:

- 1) **Piece placement** F : Contains, for each of the 64 squares, the information on whether there is a piece on it and, if so, which piece.
- 2) **Side to move** Z : There are two possibilities for Z :
 - White to move.
 - Black to move.
- 3) **Castling situation** R : For each player, there are four castling situations possible:
 - No castling possible
 - Only kingside castling possible
 - Only queenside castling possible
 - Castling possible on both sides
- 4) **En passant situation** E :
 - There is no en passant capture
 - There is an en passant capture (in this case, we must distinguish which pawn can be captured en passant)
- 5) **Move counter**: Number of moves since the last pawn move or since the last capture, whichever occurred last. This is used for the 50-move rule.
- 6) **Repetition count**: Number indicating how many times this piece distribution has occurred in the chess game so far. If the repetition count is three, the game ends in a draw.

Shannon himself defines a chess position in [4] such that it contains the information 1) to 5), although he — as we will see in Chapter 2 — only uses information from 1) in his estimate. Chinchalkar and Tromp completely ignore 5) and 6). In Chinchalkar’s case, this is found in [1] on p. 181. In Tromp’s case, this follows from the fact that he describes initial positions — “chess positions with additional information,” which we will examine in more detail in Chapter 6 — through a modified FEN notation that uniquely identifies them, and this modification overwrites the field for the move counter in the FEN notation. Thus, in his definition, initial positions and therefore also chess positions do not have a move counter. Information about the repetition number is not included in the FEN notation at all, and thus it is not part of the relevant information for a chess position in his definition. In this thesis, we follow these more modern

authors, so a chess position is defined as follows:

Definition 1.1 (Chess Position) *A chess position S is a tuple (F, Z, R, E) , where F is the piece placement, Z is the side to move, R is the castling situation, and E is the en passant situation. Two chess positions S and S' are different if they differ in one of these four components.*

Definition 1.2 (Legal and Illegal Chess Position) *A chess position S is called legal if it can occur in a chess game; otherwise, it is called illegal.*

Since we have a lot to explain over the course of this Bachelor's thesis, we will define some terms here that will help us:

- **Capture** is the term for when a piece is taken.
- The variable **caps** describes the number of captures.
- **Piece** is a collective term for the pieces knight, bishop, rook, and queen.
- **Promotion** is the term for a pawn's conversion into a piece when it reaches the opposite side of the board.
- A **piece combination** is a tuple that contains, for each piece type and each color, the number of occurrences of that piece type on the board.
- A **one-sided piece combination** is a piece combination that only contains information about the piece types of one color.
- A **legal piece combination** is a piece combination that can be reached through a chess game.
- The variables **wB**, **wD**, **wT**, **wL**, and **wS** describe (in that order) the number of white pawns, queens, rooks, bishops, and knights on the board.
- The variables **sB**, **sD**, **sT**, **sL**, and **sS** describe (in that order) the number of black pawns, queens, rooks, bishops, and knights on the board.
- The variables **wprom** and **sprom** describe the number of white or black promotions that have taken place.
- The variables **wpx** and **bpx** describe the number of white or black pawns that have been captured.
- **Pawn territory** is the collective term for the 48 squares of ranks two through seven, as these are the only squares on which pawns can stand.

2 Shannon's Estimate

In the first part of this chapter, we follow Shannon's arguments in [4]. In the second half, we will explain how Shannon arrived at his estimated value and put it in context. The second half from the explanation of the choice of the multinomial coefficient is my own work.

In his work, Shannon proves that there cannot be a computer that plays chess perfectly. He does this by showing that there are only two ways to program a computer that plays chess perfectly and that neither can ever be implemented in practice, given the vast number of chess positions and chess games.

The first possibility is a program that runs a minimax algorithm with infinite depth and then chooses the best move. For this approach, however, the program would need to go through and evaluate all possible chess games from the start of play. Shannon estimates that there are about 10^{120} different chess games, which would take a computer of his time about 10^{90} years to evaluate. Although computers today are much faster, the duration would still be far too long.

The second possibility is a directory in which, for every possible chess position, the best possible response move is stored. Shannon explains that building such a directory would theoretically be possible by asking strong chess players and analyzing positions with the minimax algorithm described above. The problem with this possibility is the estimated number of possible chess positions of about 10^{43} . This is also far too large, making the implementation of such a directory likewise impossible in practice.

Shannon arrives at the number 10^{43} through the following multinomial coefficient, which he rounds up to 10^{43} :

$$\binom{64}{32, 8, 8, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1} \approx 4,6347 \cdot 10^{42}$$

Shannon does not explain why he uses this multinomial coefficient, but exactly this multinomial coefficient calculates the number of ways of distributing the pieces that are on the board at the beginning of the game onto the 64 squares. It is highly likely that he uses this multinomial coefficient as the estimated value for the total number of possible chess positions for precisely that reason.

Here one can see well that Shannon uses only differences in piece distribution for his estimate and thus ignores possible differences in side to move, the en passant situation, the castling situation, and the move counter. One could argue that rounding $4,6347 \cdot 10^{42}$ up to 10^{43} by more than doubling it could be counted as incorporating the side to move. Shannon chooses not to incorporate further information because it is irrelevant to his argument whether there are 10^{43} , 10^{50} , or 10^{100} different legal chess positions — all are numbers of chess positions that no computer could evaluate in a reasonable time.



Fig. 1: This chess position is illegal, because the bishop on f4 can only stand there if one of the pawns on b2 or d2 had moved. But this position is counted in Shannon's estimate.

Shannon's estimate is — as we will see in Chapter 6 — a very good approximation to the number of legal chess positions for its low computational cost. However, his estimated value is not an upper bound since the en passant situation, the castling situation, and all piece distributions where at least one promotion or at least one capture occurred are not taken into account. Moreover, it is not a lower bound either, because illegal chess positions (see Fig. 1), whose number is unbounded from above, are also included.

3 Chinchalkar’s Solution

In his work [1], Shirish Chinchalkar, with the help of computer programs, finds an upper bound on the order of $1,77894 \cdot 10^{46}$.

The file *npos ORIG.c* from William Entriken’s GitHub post [2] contains the program code used by Chinchalkar. In the next chapter, we will explain the concepts used by Chinchalkar in [1] and compare them with the code instructions from [2]. Where we do not refer to the program code, all of Chinchalkar’s results are from source [1].

Chinchalkar chooses a two-step solution. In the first step, he uses necessary conditions for legal chess positions to find a set X of piece combinations that is as small as possible, containing all legal piece combinations. In the second step, for each piece combination in X , he calculates an upper bound on the number of ways of placing the pieces described by that combination on the board. Finally, he sums all these upper bounds to get an upper bound on all legal chess positions.

3.1 Calculating the Piece Combinations

In [1], Chinchalkar mentions some unproven necessary conditions with which he reduces the set X to 58,084,310 elements. In this chapter, we will formulate and prove all the conditions from Chinchalkar’s work as theorems. At the end of each proof, we look at the implementation in code. All proofs in this subsection, all lemmas, and the concepts of column switching and “capture and sacrifice columns” are my own work. To prove Chinchalkar’s conditions, we introduce some terms and prove a lemma.

Definition 3.1 (Blocked Column and Free Column) *A column (file) is blocked if in that file, two pawns of different colors block each other so that neither can promote. A column is free if it is not blocked.*

Definition 3.2 (Column Switch) *A column switch occurs when a pawn captures a piece and thus changes columns.*

Definition 3.3 (Capture Column and Sacrifice Column) *When a pawn captures a piece, the file in which the pawn was standing is called the capture file (capture column). The file in which the captured piece was standing is called the sacrifice file (sacrifice column).*

We say that pawns in free columns can promote without an additional capture because pieces merely need to be moved aside, but no further captures are necessary for these pawns to promote.

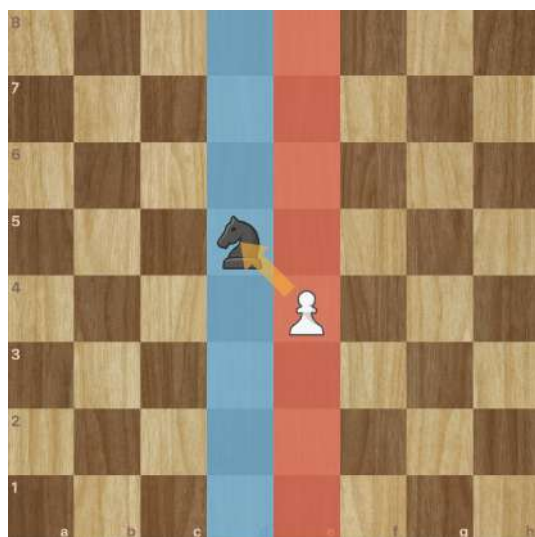


Fig. 2: The white pawn captures the black knight. The capture column is marked in red and the sacrifice column in blue.



Fig. 3: The two pawns in file c block each other, whereas the pawn in file f is in a free file and can thus promote without an additional capture.

Lemma 3.4 *It holds that:*

- a) In a move without a capture, no column becomes free.*
- b) If there has been no capture yet, no promotion can occur.*
- c) In a move in which a pawn captures a piece, at most the capture file becomes free.*
- d) In a move in which a pawn is captured, at most the capture file and the sacrifice file become free.*
- e) In a move in which a piece captures another piece, no column becomes free.*

Proof We prove the statements one by one. The following figures illustrate the various cases.

Proof (a): Free columns can only arise through column switches and thus through captures. Hence, in a move without a capture, no file can become free.

Proof (b): Since a pawn can only promote if it is in a free file, and no file is free at the start of the game, no promotion can occur before the first capture.

Proof (c): If a pawn captures a piece, at most the capture file of the pawn becomes free. The sacrifice file cannot become free, because no pawn disappears from it. All other files are unaffected by the capture.

Proof (d): If a pawn captures another pawn, at most the capture file and the sacrifice file become free, because all other files are unaffected by the capture. If a pawn is captured by another piece, analogously to (c), only the sacrifice file can become free. Hence at most the capture file and the sacrifice file can become free when a pawn is captured.

Proof (e): If a piece captures another piece, no pawn was moved. Therefore, there can be no column switch. As there is no column switch and no pawn is captured, no file can become free. $\square$

In the next four figures, columns a and b each show the situation before the capture, and columns g and h the situation after. Pawns in free files are marked green, and pawns in blocked files are marked red. The two blue arrows symbolize that the situation on the right arises from the capture, and the orange arrows show which pieces are involved in the capture or which pawns can promote.



Fig. 4: If a pawn captures a piece, at most the capture file becomes free.



Fig. 5: If a pawn captures another pawn, at most the capture file and the sacrifice file become free.



Fig. 6: If a piece captures a pawn, at most the sacrifice file becomes free.



Fig. 7: If a piece captures another piece, no file becomes free.

We can now formulate some of Chinchalkar's conditions as theorems and prove them. In [1], Chinchalkar only mentions the necessary conditions (1) to (4) for legal chess positions, although for him, statement (4) only applied to pawns captured by other pawns. I have extended the statement to include pawns captured by pieces. (5) is also my own addition.

Theorem 3.5 *It holds that:*

- 1) *Each side must have exactly one king on the board.*
- 2) *The following holds:*
 - (a) $wprom \leq 8 - wB$
 - (b) $sprom \leq 8 - sB$
- 3) *For each piece that is captured by a pawn, each side can perform at most one promotion.*
- 4) *For every pawn that is captured, the side whose pawn was captured can perform at most one promotion, and the other side can perform at most two promotions.*
- 5) *There are no other ways promotions can become possible than those described in (3) and (4).*

Proof 1) and 2) are direct deductions from the rules of chess, since exactly one king per side must always be on the board, and there can be no more promotions for a side than there are pawns of that side that have left the board. (5) was proven by points (a), (b), and (e) of Lemma 3.4.

Let BxP be a capture in which a pawn captures a piece, BxB a capture in which a pawn captures another pawn, and PxB a capture in which a piece captures a pawn. Let $wprom_X$ be the number of white promotions made possible by a capture X and $sprom_X$ the number of black promotions made possible by that capture. Let x_s be the maximum number of black promotions allowed by (3) and (4) in the game situation, and x_w the maximum number of white promotions.

We will use proof by contradiction for (3) and (4). We assume there is a game situation in which $\beta := wprom > x_w \vee sprom > x_s$. Then there must be a capture after which β holds for the first time. Let A be that capture after which β holds for the first time. Without loss of generality, assume that a black piece is the capturing piece. Then, for A , the following must hold:

- If A is a BxP , then $wprom_A > 1 \vee sprom_A > 1$
- If A is a BxB , then $wprom_A > 1 \vee sprom_A > 2$
- If A is a PxB , then $wprom_A > 1 \vee sprom_A > 2$

If this is not the case, then either β would not hold, or A would not be the first capture after which β holds.

Case 1: A is a BxP :

By Lemma 3.4, at most the capture file of A (call it S) becomes free. If S was free before A , then A does not make any column free, and thus no promotions

result from A . If S was blocked before A , S only becomes free if no other black pawns stand in S . Hence the black capturing pawn is the only black pawn that can, through A , reach a free file and $sprom_A \leq 1$.

Thus, in the game situation, $sprom > x_s$ cannot hold. Suppose $sprom > x_s$ held. Then $sprom > x_s$ would have had to hold already before A , because $sprom_A \leq 1$ and one black promotion also becomes possible from a capture of type BxP according to (3). This contradicts the statement that β holds for the first time after A .

Since $sprom_A > 1$ does not hold, $wprom_A > 1$ must hold. Suppose $wprom_A = n \geq 2$. Then, because S is the only file that becomes free from A , n white pawns must be in S in order for $wprom_A = n$ to hold. For that to happen, there must have been at least $n - 1$ column switches into S , so at least $n - 1$ captures $C_1, \dots, C_l$ with $l \geq n - 1$, in which white pawns captured a piece in S . Consider such a capture $B \in \{C_1, \dots, C_l\}$. There are two possibilities:

Case 1.1: B is a BxP :

Since White is capturing, all promotions enabled by B in the capture file of B are black promotions. Because the sacrifice file of B is the file S that only becomes free through A , $wprom_B = 0$. Thus B enables one less white promotion than would be allowed for a capture of type BxP according to (3).

Case 1.2: B is a BxB :

At most the capture file of B becomes free, since the sacrifice file of B is S and only becomes free through A . Because White is capturing, all promotions that become possible through B in the capture file of B are black promotions. Hence $wprom_B = 0$. This means B enables two fewer white promotions than is allowed by (4) for a capture of type BxB .

Therefore, $n - 1$ of these captures make at least $n - 1$ fewer white promotions possible than would be allowed by (3) or (4). Because A enables at most $n - 1$ more white promotions than allowed by (3) for a capture of type BxP , the captures $A, C_1, \dots, C_l$ together do not enable more white promotions than permitted by (3) and (4).

Hence it follows that $wprom > x_w$ cannot hold. If $wprom > x_w$ did hold, it would already have held before captures $A, C_1, \dots, C_l$, which contradicts the claim that β first holds after A .

It is therefore impossible for A to be a capture of type BxP .

Case 2: A is a BxB :

At most the capture file of A (S) and the sacrifice file (O) become free. Two cases are possible:

Case 2.1: $sprom_A > 2$:

Let $sprom_A = m \geq 3$. Then there must be m black pawns in O . This requires at least $m - 2$ column switches into O . The proof can then proceed analogously to Case 1, and it follows that $sprom > x_s$ either does not hold or already held before A .

Case 2.2: $wprom_A > 1$:

This case can be proven analogously to Case 2.1.
Thus it is impossible for A to be a capture of type BxB .

Case 3: A is a PxB :

A capture of type PxB frees at most the sacrifice file of A (call it O), since no pawn leaves the capture file. Since we assumed that the capturing piece is black, O becomes free when a white pawn is captured. If O becomes free because of A , then $wprom_A = 0$; if O does not become free, we also have $wprom_A = 0$. Since $wprom_A > 1$ does not hold, $sprom_A > 2$ must hold. Let $sprom_A = m \geq 3$. From here, the proof proceeds analogously to Case 2.1. Finally, it follows that $sprom > x_s$ cannot hold or must already have held before A .

Therefore, it is impossible for A to be of type PxB . Hence there is no capture A after which β holds for the first time, and since β does not hold at the start of the game, there is no game situation in which $wprom > x_w \vee sprom > x_s$. $\square$

Remark In the proof, we used the statement that file S must be blocked because it only becomes free later through capture A . This formulation has simplified the situation a bit. Of course, it is possible for a file S to be free, then become blocked by a column switch, and then become free again through a capture A . However, all pawns that were in that file when it was free for the first time do not count for promotions that the capture A makes possible, because those pawns already had a chance to promote once. Since we assumed in the proof that A enables n white promotions, the $n - 1$ additional white pawns must have arrived in file S at a time when S was blocked.

From Theorem 3.5, we derive the necessary conditions used by Chinchalkar in his program code.

Theorem 3.6 *Let*

- $\triangleright$ $wpcx$ be the number of white pieces that were captured
- $\triangleright$ $bpcx$ be the number of black pieces that were captured

Then

- a) $wprom \leq 2 \cdot bpx + wpcx + bpcx + wpx$
- b) $sprom \leq 2 \cdot wpx + wpcx + bpcx + bpx$

Proof (a) and (b) are analogous statements for the two sides, so it suffices to show (a). As we saw in Theorem 3.5, each captured black pawn can allow at most two white promotions, so in total $2 \cdot bpx$ white promotions. For each captured piece, at most one white promotion is possible, so $wpcx + bpcx$ additional white

promotions. For each captured white pawn, at most one white promotion is possible, so wpx more. Since these (according to Theorem 3.5) are the only ways to create white promotions, $2 \cdot bpx + wpcx + bpcx + wpx$ is an upper bound on $wprom$. $\square$

We collect some basic properties about the number of pieces in the following lemma.

Lemma 3.7 *It holds that:*

- $\triangleright wB, sB \in \{0, \dots, 8\}$
- $\triangleright wD, sD \in \{0, \dots, 9\}$
- $\triangleright wS, sS, wL, sL, wT, sT \in \{0, \dots, 10\}$
- $\triangleright wB + wD + wS + wL + wT + 1 \leq 16$
- $\triangleright sB + sD + sS + sL + sT + 1 \leq 16$

Proof All statements follow directly from the rules of chess. $\square$

Looking now at the implementation of our results in the *npos ORIG.c* file from [2]: Chinchalkar goes through the number of pawns, queens, rooks, knights, bishops, and pawns one side can have via the for-loops in lines 215 to 246 in the main function. The if-statement in lines 225ff ensures that (2) of Theorem 3.5 is satisfied. If it is, the one-sided piece combination is stored in the array *pieces* (lines 227-234). The array *pieces* thus represents the set of all possible one-sided piece combinations. The for-loops starting at lines 256 and 269 iterate over the array *pieces* for White and Black, respectively, going through all combinations of two one-sided piece combinations. Lines 286 to 289 check the conditions of Theorem 3.6, where $nWcaptures = wpx + wpcx$ and $nBcaptures = bpx + bpcx$. If a piece combination fulfills all criteria, it is an element of X , and the variable *nsets* is incremented by one in line 303. *nsets* thus describes the cardinality of X . As a result, the set X has 58,084,310 different elements.

3.2 Placing the Piece Combinations

In this section, we follow Chinchalkar as he determines, for each piece combination $x \in X$, a number Θ_x . Θ_x is an upper bound on the number of ways of placing the pieces that this piece combination x describes onto the board, also taking into account castling situations, en passant situations, and side to move. Summing these upper bounds gives an overall upper bound O on the number of legal chess positions. When we say that an upper bound takes en passant situations, castling situations, and side to move into account, this means that we count elements from all the cases that can occur for these factors.

For each piece combination, Chinchalkar first arranges the pawns on the board. He formulates the following theorem in [1], giving the upper bound W_{Bauer} .

Theorem 3.8 *Let*

- i be the number of white pawns in the fourth rank
- j be the number of white pawns in the fifth rank
- l be the number of black pawns in the fourth rank
- k be the number of black pawns in the fifth rank
- $p = \min(sB, 8 - j)$
- $q = \min(sB - k, 8 - i)$

Then W_{Bauer} is an upper bound on the number of ways to place wB white pawns and sB black pawns on the board in a way that considers the en passant situation and side to move, where

- $$W_{Bauer} = \sum_{i=0}^{wB} \sum_{j=0}^{wB-i} \sum_{k=0}^p \sum_{l=0}^q W_{ij} \cdot B_{ijkl} \cdot E_{ijkl}$$
- $$W_{ij} = \binom{8}{i} \cdot \binom{8}{j} \cdot \binom{32}{wB-i-j}$$
- $$B_{ijkl} = \binom{8-j}{k} \cdot \binom{8-i}{l} \cdot \binom{32-(wB-i-j)}{sB-k-l}$$
- $$E_{ijkl} = (1 + \min(2l, i) + 1 + \min(2j, k))$$

Proof All of the explanations except for the explanations of the terms $\min(2j, k)$ and $\min(2l, i)$ are my own work.

W_{ij} describes the number of ways of placing wB white pawns on the board such that i white pawns are on the fourth rank and j white pawns are on the fifth rank. Once these wB white pawns have been placed, B_{ijkl} describes the number of ways of placing the sB black pawns on the remaining squares so that k black pawns are on the fifth rank and l black pawns are on the fourth rank. In these binomial coefficients, 32 is used because those pawns can only occupy the 32 squares outside the fourth and fifth ranks.

In E_{ijkl} , Chinchalkar accounts for both side to move and the possible en passant situations for this pawn distribution. The term $1 + \min(2j, k)$ is the maximum number of en passant situations if White can capture en passant, and the term $1 + \min(2l, i)$ is the maximum number of en passant situations if Black can capture en passant. If there are two different en passant situations, that means two different pawns can be captured en passant. Assume White can capture en passant. The argument for $1 + \min(2l, i)$ is analogous. For calculating an upper bound on the number of possible en passant situations, Chinchalkar needs to

figure out how many different black pawns can be captured en passant in this pawn distribution. If j white pawns are on the fifth rank, at most $2j$ different black pawns can be captured en passant, because each white pawn can capture only left or right. Furthermore, these black pawns must also be on the fifth rank, so we take $\min(2j, k)$ as an upper bound for the number of en passant situations when White can capture en passant. Another en passant situation exists if there is no en passant capture possible. Therefore $\min(2j, k) + 1$ is an upper bound on the number of en passant situations when White is to move.

Summing the product $W_{ij} \cdot B_{ijkl} \cdot E_{ijkl}$ over all possible values of i, j, k, l yields an upper bound on the number of ways of placing wB white pawns and sB black pawns on the board, taking into account the en passant situation and side to move. $\square$

Although W_{Bauer} restricted the number of possible pawn placements, Chinchalkar points out that W_{Bauer} still overestimates the number of pawn positions in two ways:

1. Not all pawn positions are possible. (see Fig. 8)
2. Not all en passant situations are possible (see Fig. 9)



Fig. 8: It is impossible for white pawns to stand simultaneously on a2, a3, and b2, because either the a2-pawn or the b2-pawn must have moved to a3.



Fig. 9: It is impossible for the white pawn on e5 to capture the black pawn on h5 en passant. However, W_{Bauer} counts this possibility.

In order to count as few illegal pawn placements as possible, Chinchalkar mentions a way to improve W_{Bauer} by determining how many files are blocked based on the number of captures. Even though he does not discuss this idea further in his paper, it is found in lines 137 to 152 of the *npos ORIG.c* file. We will present the calculation as Theorem 3.9.

Theorem 3.9 *If $x \in \{1, \dots, 8\}$ files are blocked, then M is an upper bound on the number of ways of placing wB white pawns and sB black pawns on the board, taking into account the en passant situation and side to move, with*

$$M = \binom{8}{x} \cdot 15^x \cdot \binom{48 - 2x}{wB - x} \cdot \binom{48 - wB - x}{sB - x} \cdot y$$

where $y = (1 + \min(2 \cdot wB, sB) + 1 + \min(2 \cdot sB, wB))$

For $x = 0$, we continue to use W_{Bauer} from Theorem 3.8. Hence $M = W_{Bauer}$ in this case.

Proof These explanations combine code comments and my own work.

Let $x \in \{1, \dots, 8\}$ files be blocked. Then there are at most $\binom{8}{x}$ ways to choose which files are blocked. In each blocked file, there are at most 15 ways in which a white and a black pawn can block each other: (2,3), (2,4), (2,5), (2,6), (2,7), (3,4), (3,5), (3,6), (3,7), (4,5), (4,6), (4,7), (5,6), (5,7), (6,7), where the first component is the rank of the white pawn and the second the rank of the black

pawn (see Figs. 10 and 11). In a blocked file, there can also be more than one white and one black pawn; we will consider these additional pawns shortly when distributing the remaining pawns. Altogether, there are at most 15^x ways to position these x white and x black pawns in the x blocked files.

We then distribute the rest of the pawns on the board. Since there are at most 48 squares where a pawn can stand and $2x$ pawns are already placed, there are still at most $\binom{48-2x}{wB-x}$ ways to distribute the $wB-x$ remaining white pawns. For the $sB-x$ remaining black pawns, there are $48-wB-x$ squares left, giving at most $\binom{48-wB-x}{sB-x}$ possibilities. The factor y is known from Theorem 3.8, except that here we use the total number of white or black pawns instead of the number of white or black pawns in the fourth or fifth rank. This change does not make y any smaller than E_{ijkl} , so the proof of y is analogous to the proof for E_{ijkl} in Theorem 3.8. Therefore, y accounts for possible en passant situations and side to move, and thus M is indeed the upper bound. $\square$



Fig. 10: The pawn blockades (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (3, 4), (3, 5), and (3, 6).



Fig. 11: The pawn blockades (3, 7), (4, 5), (4, 6), (4, 7), (5, 6), (5, 7), and (6, 7).

After the pawns, Chinchalkar places the white rooks and the white king on the board.

Theorem 3.10 W_{wK} is an upper bound on the number of ways of placing the white king and wT white rooks on the S available squares of the board, taking into account the possible castling situations, where

$$W_{wK} = \binom{S-1}{wT} \cdot S + \binom{S-1}{wT} \cdot f \quad \text{with} \quad f = \begin{cases} 1 & \text{if } wT = 0 \\ 3 & \text{if } wT = 1 \\ 4 & \text{if } wT \geq 2 \end{cases}$$

Proof This proof is from [1] and has been presented here in a bit more detail. When placing the king, two cases can arise. We derive an upper bound for each case and then add them.

Case 1: The king is not on e1: A castling is impossible because the king would have moved. First we place the king, which is at most S ways. Then the number of ways to position the wT rooks is given by the binomial coefficient $\binom{S-1}{wT}$. Hence $\binom{S-1}{wT} \cdot S$ is an upper bound for this case.

Case 2: The king is on e1: By assumption, there is exactly one way to place the king. Because the king is still on its start square, castling is basically not excluded. The variable f describes how many different castling situations exist for White. If there are no rooks ($wT = 0$), the only possible castling situation is not castling ($f = 1$). If there is one rook ($wT = 1$), we can have $f = 3$ castling situations: only kingside castling, only queenside, or none. For $wT \geq 2$, we have

the same three situations plus the possibility of castling on both sides ($f = 4$). For placing the rooks, there are at most $\binom{S-1}{wT}$ ways. Thus $\binom{S-1}{wT} \cdot f$ is the upper bound for this case.

Hence $W_{wK} = \binom{S-1}{wT} \cdot S + \binom{S-1}{wT} \cdot f$ is indeed the upper bound. $\square$

Chinchalkar notes that an analogous upper bound W_{sK} for placing the black king and black rooks can be found, which we will call W_{sK} .

The remaining piece groups are then placed one by one on the board. The number of ways to do this can be calculated by a multinomial coefficient. Altogether, for a piece combination

$$x = (wB, wD, wT, wL, wS, sB, sD, sT, sL, sS),$$

the upper bound Θ_x is calculated as follows:

$$\Theta_x = M \cdot W_{wK} \cdot W_{sK} \cdot \binom{62 - wB - sB - wT - sT}{wD, wL, wS, sD, sL, sS}$$

By summing Θ_x over all piece combinations $x \in X$, Chinchalkar obtains the upper bound $O \approx 1,77894 \cdot 10^{46}$ for the number of legal chess positions.

If we look at the implementation of these results in the code *npos ORIG.c*: The function **arrange** (lines 123-194) takes a piece combination x as input and returns Θ_x as output. The upper bound W_{Bauer} from Theorem 3.8 is computed for various values of wB and sB in the **place_pawns** function (lines 64-95) and stored in the array **pawns**. The upper bound M from Theorem 3.9 is computed in lines 130 to 145. The function **place_rooks** (lines 106-121) computes the upper bound W_{wK} from Theorem 3.10 by using the number of rooks (variable **r**) and the number of remaining free squares (variable **l**) as input. Likewise, the upper bound W_{sK} is computed with **place_rooks**. In lines 160 to 172, we use multinomial coefficients to calculate the distribution of the remaining pieces. The program outputs $O = 1,77894 \cdot 10^{46}$ as the result.

4 Tromp’s Solution

In 2022, Tromp was able to beat Chinchalkar’s upper bound with the number

$$8.726.713.169.886.222.032.347.729.969.256.422.370.854.716.254 \approx 8,73 \cdot 10^{45}.$$

The program code with which he computed this upper bound can be found in the file *CountChess.hs* of his GitHub post [5]. Because I encountered runtime issues when I wanted to modify Tromp’s Haskell code, I translated his program code into C and wrote my extension based on that translation (see Appendix). While translating, I made sure that all function values, intermediate results, and final results of the original and of the translation match exactly. Because it is much more likely that the reader understands C code, we will point to the C code. [5] is the only one of Tromp’s works that I refer to here.

Tromp also divides his solution into two steps. First, he filters out as many illegal piece combinations as possible and then, for each remaining piece combination, he computes an upper bound on the number of ways of placing those pieces on the board in dependence on castling and en passant situations. Finally, he sums all upper bounds and multiplies the result by some multipliers. We will formulate and prove all concepts used by Tromp as theorems and lemmas in this chapter.

All conditions and concepts used in this chapter come from Tromp’s code and were basically explained in his comments. However, he leaves it to the reader to check that these conditions are valid. Therefore, all the proofs in this chapter are my own.

4.1 Fixed Pieces and Multipliers

There are major differences between Tromp’s and Chinchalkar’s solutions regarding the implementation of side to move, en passant situation, and castling situations. We will go through Tromp’s approach step by step, beginning with side to move.

In each of his calculations, Tromp assumes White to move and only multiplies the final result by two (line 272) to also account for all chess positions in which Black is to move. We will likewise assume White to move in the next two chapters. All statements can be proven analogously if Black is to move.

To implement castling situations, Tromp introduces the concepts of a fixed rook and a fixed king. The rules of chess state that a side may only castle if neither the king nor the involved rook has moved yet. It follows that the castling situation in place determines which rooks could not have moved yet. Rooks that, due to the present castling situation, could not have moved yet, we call fixed rooks. As soon as a side has at least one fixed rook, the king must also be on its initial square, since otherwise castling would be impossible. Such a king is called a fixed king.

Definition 4.1 (Fixed Rook) A rook is called *fixed* if the present castling situation demands that this rook could not have moved yet. The number of white fixed rooks is $fwr \in \{0, 1, 2\}$, and the number of black fixed rooks is $fbr \in \{0, 1, 2\}$.

Definition 4.2 (Fixed King) A king is called *fixed* if its side has at least one fixed rook. The number of fixed white kings is $fixwk \in \{0, 1\}$, and the number of fixed black kings is $fixbk \in \{0, 1\}$.

The following lemma holds analogously for Black with fbr .

Lemma 4.3 *White can...*

- a) *no longer castle* $\Leftrightarrow fwr = 0$
- b) *only castle queenside or only castle kingside* $\Leftrightarrow fwr = 1$
- c) *castle on both sides* $\Leftrightarrow fwr = 2$

Proof (a), (b), and (c) follow directly from the definition of a fixed rook. $\square$

While $fwr \in \{0, 2\}$ has only one way to place the fixed white rooks, for $fwr = 1$ the fixed rook can be either on a1 (for queenside castling) or on h1 (for kingside castling). The multiplier $multFWR$ indicates how many ways there are to place the fwr white fixed rooks. We have:

$$multFWR = \begin{cases} 1 & \text{for } fwr \in \{0, 2\} \\ 2 & \text{for } fwr = 1 \end{cases}$$

For Black, $multFBR$ is defined analogously to $multFWR$.

Tromp implements the en passant situation by distinguishing whether White can capture en passant on this move or not. If White can capture en passant, there must be both a white pawn on the fifth rank and a black pawn that just moved to the fifth rank on the previous move. Since we will later address how many ways there are to place these two pawns, we can assume for now that each stands in a specific file, so its square is fixed.

Definition 4.4 (Fixed Pawn) *A pawn is called fixed if the en passant situation requires that the pawn stand on a specific square. The number of fixed pawns per side is $fixp \in \{0, 1\}$. A file is called a fixed file if it contains a fixed pawn.*

Moreover, the en passant situation requires that the black fixed pawn made the last move.

From now on, let wp be the number of non-fixed white pawns, and bp the number of non-fixed black pawns. The following lemma shows that the value of $fixp$ indicates the present en passant situation.

Lemma 4.5 *White can...*

- a) *not capture en passant* $\Leftrightarrow fixp = 0$
- b) *capture en passant* $\Leftrightarrow fixp = 1$

Proof We prove (a); (b) can be proven analogously.

$\Rightarrow$: If White cannot capture en passant, no pawns need to stand on specific squares to enable an en passant capture, hence $fixp = 0$.

$\Leftarrow$: If $fixp = 0$, the en passant situation does not require that any pawns be on specific squares. If an en passant capture were possible, we would need pawns on certain squares. Thus White cannot capture en passant. $\square$

As noted in the definition, the number of fixed white pawns is always equal to the number of fixed black pawns. One must be careful here not to confuse the notion of “fixed white pawns” with white pawns that can capture en passant. There exist positions in which White can capture a black pawn en passant with two different pawns (see Fig. 12). But there is always only one fixed white pawn, because only one white pawn must stand on a specific square for a possible en passant capture, and the en passant situation does not require a second white pawn on a specific square.



Fig. 12: Black's last move was a pawn from d7 to d5. Both the white pawn on c5 and the white pawn on e5 can capture the black pawn on d5 en passant.

We mentioned that later we will determine how many ways there are to place the fixed pawns on the fifth rank. This number is given by the multiplier $multEP$.

Lemma 4.6 *It holds that:*

$$multEP = \begin{cases} 1 & \text{if } fixp = 0 \\ 14 & \text{if } fixp = 1 \end{cases}$$

Proof We prove case by case:

Case $fixp = 0$: There is exactly one way to place zero fixed pawns in the eight files.

Case $fixp = 1$: If the black fixed pawn stands in files b, c, d, e, f, or g, then the white fixed pawn must stand either on its left or right side to capture it en passant. If the black fixed pawn stands in file a, the white fixed pawn must stand on b, and if the black fixed pawn stands on h, the white fixed pawn must stand on g. Altogether, there are therefore at most 14 ways. $\square$

Because the black fixed pawn is the one that made the last move, Tromp, unlike Chinchalkar (who used Theorem 3.8), does not need to determine how many pawns can be captured en passant by White. There is exactly one black fixed pawn that can be captured en passant, accounted for by the multiplier $multEP$ also covering all possible positions of the black fixed pawn.

4.2 Implementing the Piece Combinations

Tromp’s code begins by iterating over all possible values of fwr , fbr , and $fixp$ with loops. In the innermost loop, it calls the `count` function (line 264), which in turn computes an upper bound on the number of legal chess positions, given fwr , fbr , and $fixp$. The multipliers $multFWR$, $multFBR$, and $multEP$ are defined in lines 252, 256, and 261, and multiplied into the result in line 264.

Tromp’s calculation of piece combinations differs in several respects from Chinchalkar’s because of the introduction of fixed pawns, rooks, and kings. Because the number of fixed pawns and rooks is already known as input parameters to the `count` function, Tromp uses the number of non-fixed pawns and rooks. He also calculates how many fixed kings each side has, because that helps reduce the number of available squares. We will define wp and bp here, as we will need them often later on. Let wp be the number of non-fixed white pawns, and bp be the number of non-fixed black pawns.

Tromp also filters out some illegal piece combinations. In his code, this happens right in the loop conditions by including the number of pawns still available for promotions, using auxiliary variables like $wnp1$, so that he does not have to check at the end if too many promotions took place. As further filters, he uses these two conditions:

Lemma 4.7 *It holds that:*

$$a) \quad bpx + caps - wproms \geq 0$$

$$b) \quad wpx + caps - bproms \geq 0$$

Proof We prove (a); (b) is shown analogously. Rewriting (a) yields $bpx + caps \geq wproms$. Let gx be the number of captures in which no black pawn was captured, then $caps = gx + bpx$, so $2 \cdot bpx + gx \geq wproms$. From Theorem 3.5, each captured black pawn can allow at most two white promotions, and each other captured piece can allow at most one white promotion. Hence White can make at most $2 \cdot bpx + gx$ promotions, so $bpx + caps - wproms \geq 0$ must hold. $\square$

Unlike Chinchalkar, Tromp never calculates the exact number of piece combinations. A comparison of such numbers would have little meaning anyway, since Tromp uses many more piece combination settings for the different $fixp$, fwr , and fbr .

4.3 Calculating the Pawn Distribution

In this subsection, we look at how Tromp calculates the number of ways to place a piece combination on the board, given $fixp$, fwr , and fbr . Since fixed pawns and rooks must stand on specific squares, they are placed on the board first.

Then the pawns are placed. The calculation of an upper bound on the number of pawn distributions is where Tromp's and Chinchalkar's solutions differ most significantly. We begin by introducing the variables *maxuwp* and *minopp*:

Lemma 4.8 *Define the variables *maxuwp* and *minopp* by:*

$$\triangleright \text{maxuwp} := \text{bpx} + \text{caps} - \text{wproms}$$

$$\triangleright \text{minopp} := \max(0, \text{fixp} + \text{wp} - \text{maxuwp})$$

Then:

a) *maxuwp* is the maximum number of white pawns in free files.

b) *minopp* is the minimum number of blocked files.

Proof First we prove (a) and then use (a) to prove (b).

Proof (a): From Lemma 4.7, we have $\text{bpx} + \text{caps}$ as the maximum number of white promotions. Because wproms is the actual number of white promotions, $\text{bpx} + \text{caps} - \text{wproms}$ is the maximum number of white promotions still possible without another capture. Exactly those pawns in free files can promote without an additional capture. Hence (a).

Proof (b): If at most *maxuwp* white pawns stand in free files, then at least $\max(0, \text{fixp} + \text{wp} - \text{maxuwp}) = \text{minopp}$ white pawns stand in blocked files. We claim that these *minopp* white pawns cannot stand in fewer than *minopp* files, which shows that *minopp* is the minimum number of blocked files.

We prove the claim by contradiction. Suppose there are exactly y white pawns in $y - z$ blocked files, with $y - z < \text{minopp}$ and $y \geq \text{minopp}$. Since at most $y - z$ white pawns would stand in the $y - z$ blocked files without a column switch, there must be at least z column switches and thus at least z captures in which a white pawn took a piece in one of these $y - z$ blocked files. Every time a white pawn captures a piece in a blocked file, it eliminates at least one white promotion that would have been possible if that capture had used a free (or becoming-free) file as the sacrifice file. That is because a white pawn in a blocked file cannot promote without an additional capture, whereas a pawn in a free (or becoming-free) file can.

Therefore, in this situation, there are at most $\text{bpx} + \text{caps} - z$ white promotions. Hence at most $\text{bpx} + \text{caps} - \text{wproms} - z$ white pawns can be in free files, so at least $\text{minopp} + z$ white pawns must be in blocked files. Since $\text{minopp} + z > y$, this contradicts the assumption that there are exactly y white pawns in fewer than *minopp* blocked files. Because $y \geq \text{minopp}$, it is impossible for at least *minopp* white pawns to be distributed over fewer than *minopp* files. $\square$

In the code, *maxuwp* and *minopp* are computed in lines 210 and 218. The more crucial of the two variables is *minopp*, since from now on we can assume that

there are at least *minopp* blocked files. This assumption helps us drastically reduce the number of possible pawn distributions. Another helpful concept is introduced by the definition of an enclosed square.

Definition 4.9 (Enclosed Square) *A square ij is enclosed if there exist squares iw and is with $i \in \{a, \dots, h\}$ and $j, w, s \in \{1, \dots, 8\}$ such that the following conditions hold:*

- $w < j < s$
- there is a white pawn on iw and a black pawn on is
- all squares ik with $k \in \{w + 1, \dots, s - 1\}$ are not occupied by pawns

We also say that pawns enclose the square ij .

Only squares in blocked files can be enclosed, because otherwise the first item in the definition would not be satisfied.



Fig. 13: This figure illustrates the concept of enclosed squares by examples.

File a: There is no black pawn facing the white pawn, so there can be no enclosed squares in that file.

File b: Although white and black pawns face each other, all squares between them are occupied by pawns, so there are also no enclosed squares here.

File c: Although no pawns stand between the black and the white pawn, the white pawn is on a higher rank than the black pawn, so there is also no enclosed square here.

File d: Now all conditions are fulfilled for the first time, and square d4 is an enclosed square.

File e: The square e4 is also enclosed, because a rook is not a pawn and is accepted in the pawn territory. Other pieces in the pawn territory are fine because they will be placed on the board later and do not affect the pawn distribution.

File f: Only the square f5 is enclosed, because the pawn on f4 prevents all squares between the pawns on f2 and f6 from being free of pawns.

Having developed a good understanding of enclosed squares, we can now look at the following theorem.

Theorem 4.10 *Let $Z_{p,s,n}$ be the number of ways to enclose s squares with pawns in p blocked files and choose those p blocked files from n available files. Let $p \leq n$. Then $Z_{p,s,n} = m(p, s, n)$, where*

$$m(p, s, n) = \begin{cases} y & \text{if } p > 0 \\ 1 & \text{if } p = 0 \text{ and } s = 0 \\ 0 & \text{if } p = 0 \text{ and } s > 0 \end{cases}$$

and

$$y = m(p, s, n-1) + \sum_{i=0}^{\min(s,4)} (5-i) \cdot m(p-1, s-i, n-1).$$

Proof We prove the claim by complete induction on n , the number of available files.

Base case: $n = 0$

Case $p = s = 0$: We have $Z_{0,0,0} = 1$, because there is exactly one way to enclose zero squares in zero blocked files and choose zero blocked files from zero available ones. Thus $Z_{0,0,0} = m(0, 0, 0)$ since by definition $m(0, 0, 0) = 1$.

Case $p > 0$: Does not occur for $n = 0$ because $p \leq n$.

Case $p = 0$ and $s > 0$: By definition, $m(0, s, 0) = 0$. Also $Z_{0,s,0} = 0$, because it is impossible to enclose more than zero squares with pawns in zero blocked files chosen from zero available files.

Inductive hypothesis: For a fixed but arbitrary $n \in \mathbb{N}$, the claim of Theorem 4.10 holds for all $k \in \{0, \dots, n\}$.

Inductive step: $n \Rightarrow n+1$

Let $X_{p,s}$ be the number of ways to enclose s squares in p blocked files, for any fixed n . Then for all n :

$$Z_{p,s,n} = X_{p,s} \cdot \binom{n}{p}.$$

We multiply $X_{p,s}$ by the number of ways to choose which p files from the n available are blocked, obtaining $Z_{p,s,n}$. Hence:

$$Z_{p,s,n+1} = \frac{\binom{n+1}{p}}{\binom{n}{p}} \cdot Z_{p,s,n} = \frac{n+1}{n-p+1} \cdot Z_{p,s,n}.$$

Since $p \leq n$, the fraction is defined for all p and n . This equality will be useful for the various cases.

Case $p = s = 0$: By definition, $m(0, 0, n+1) = 1$. By the inductive hypothesis, $Z_{0,0,n} = 1$, and for $p = 0$ we apply (2) to get $Z_{0,0,n+1} = 1$.

Case $p = 0$ and $s > 0$: By definition, $m(p, s, n) = m(p, s, n+1) = 0$. By the inductive hypothesis, $Z_{p,s,n} = 0$ and thus from (2) we again get $Z_{p,s,n+1} = 0$.

Case $p > 0$: We need to show

$$Z_{p,s,n+1} = m(p, s, n) + \underbrace{\sum_{i=0}^{\min(s,4)} (5-i) \cdot m(p-1, s-i, n)}_{=:A}.$$

Since by the inductive hypothesis $Z_{p,s,n} = m(p, s, n)$, from (2) we have:

$$m(p, s, n) = \frac{n-p+1}{n+1} \cdot Z_{p,s,n+1}.$$

Moreover, by the inductive hypothesis $m(p-1, s-i, n) = Z_{p-1,s-i,n}$ for all $i \in \{0, \dots, s\}$. Hence:

$$A = \sum_{i=0}^{\min(s,4)} (5-i) \cdot Z_{p-1,s-i,n} = \binom{n}{p-1} \cdot \sum_{i=0}^{\min(s,4)} (5-i) \cdot X_{p-1,s-i}.$$

We now look at the sum more closely. $(5-i)$ indicates how many ways there are to enclose i squares in a blocked file with pawns (see Figs. 14-16). Hence $(5-i) \cdot X_{p-1,s-i}$ indicates how many ways there are to enclose i squares in one blocked file and $s-i$ squares in $p-1$ other blocked files. Summing over all possible numbers of squares enclosed by a file yields:

$$X_{p,s} = \sum_{i=0}^{\min(s,4)} (5-i) \cdot m(p-1, s-i, n).$$

Because equation (1) holds for all n , it also holds for $Z_{p,s,n+1}$. Rearranging gives:

$$X_{p,s} = \frac{Z_{p,s,n+1}}{\binom{n+1}{p}}.$$

So we obtain:

$$A = \frac{\binom{n}{p-1}}{\binom{n+1}{p}} \cdot Z_{p,s,n+1} = \frac{p}{n+1} \cdot Z_{p,s,n+1}.$$

Putting it all together:

$$m(p, s, n+1) = m(p, s, n) + A = \left(\frac{n-p+1}{n+1} + \frac{p}{n+1} \right) \cdot Z_{p,s,n+1} = Z_{p,s,n+1}.$$

□

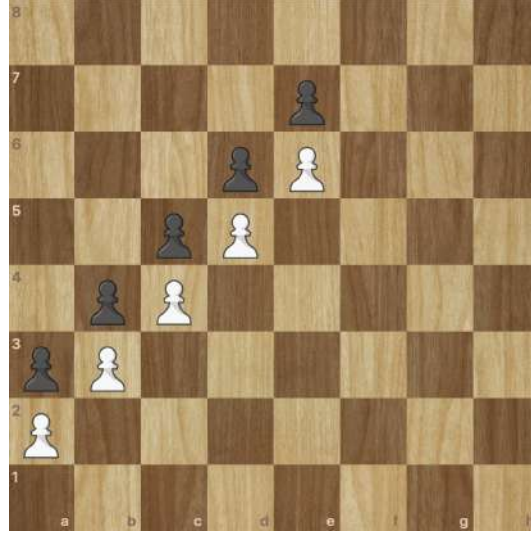


Fig. 14: All ways to enclose zero squares in one file with pawns.



Fig. 15: Left: all ways to enclose one square; right: all ways to enclose two squares in one file with pawns.

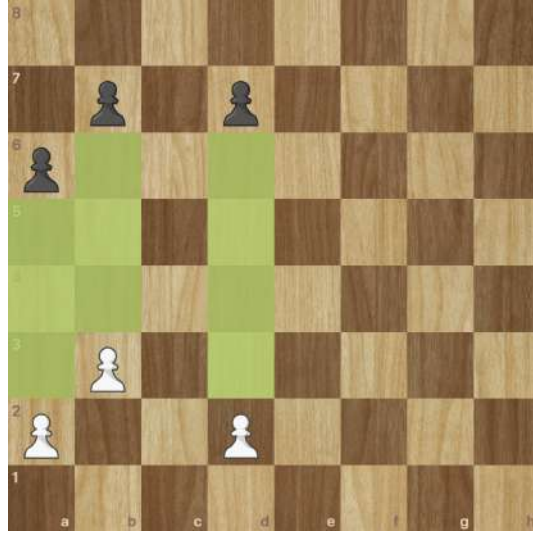


Fig. 16: Left: all ways to enclose three squares; right: all ways to enclose four squares in one file with pawns.

In the code, the values of $m(p, s, n)$ are computed by the functions `opps` and `moppsfunction` and stored in the array `mopps` (lines 45-70). Using Theorem 4.10, we can now find an upper bound on the number of possible pawn distributions when en passant capture is not possible.

Theorem 4.11 $B_{0,wp,bp}$ is an upper bound on the number of ways to place wp non-fixed white pawns and bp non-fixed black pawns on the board if en passant capture is not possible, where

$$B_{0,wp,bp} = \sum_{s=0}^{4 \cdot \text{minopp}} m(\text{minopp}, s, 8) \cdot \binom{48 - 2 \cdot \text{minopp} - s}{wp - \text{minopp}, bp - \text{minopp}}.$$

Proof By Lemma 4.8, each possible pawn distribution has at least minopp blocked files, so we can substitute minopp into $m(p, s, n)$. Each summand calculates the number of ways to place the wp non-fixed white pawns and bp non-fixed black pawns on the board under the condition that exactly s squares are enclosed by pawns in those minopp blocked files. $m(\text{minopp}, s, 8)$ is the number of ways to place minopp white pawns and minopp black pawns in the blocked files, and the multinomial coefficient is the number of ways to place the remaining pawns. In each blocked file, between zero and four squares can be enclosed, so the maximum number of enclosed squares in minopp blocked files is $4 \cdot \text{minopp}$. Summing over all possible values of s yields an upper bound on the number of ways to place wp non-fixed white pawns and bp non-fixed black pawns on the board if en passant capture is not possible. $\square$

Remark Theorem 4.11 also includes all pawn distributions with more than *minopp* blocked files, because the remaining pawns can also be arranged so that more files become blocked. A “remaining pawn” can also stand in a blocked file, but not on an enclosed square.

The upper bound from Theorem 4.11 is computed in the first part of the function `pawnsfunction` (lines 96-110) and stored in the array `pawns`.

Now we consider the case where en passant capture is possible, i.e. $fixp = 1$. Let us call the file containing the white fixed pawn the white fixed file and the one containing the black fixed pawn the black fixed file. As these files already contain the fixed pawns, they cannot be treated as “normal” blocked or free files, and must be considered separately. Thus we go through all the cases that can occur in these files. The variable $dw \in \{0, 1\}$ indicates whether the black fixed file is blocked ($dw = 1$) or free ($dw = 0$). Analogously, $db \in \{0, 1\}$ indicates whether the white fixed file is blocked or free. Further, the variable $ds1 \in \{0, 1, 2\}$ describes how many squares in the black fixed file are enclosed by pawns, and $ds2 \in \{0, 1\}$ how many squares in the white fixed file are enclosed. For clarity, see the figures on the following pages.

For the black fixed file, the following cases $(dw, ds1)$ are possible:

- Case (0, 0): free file
- Case (1, 0): blocked file, no enclosed square
- Case (1, 1): blocked file, one enclosed square
- Case (1, 2): blocked file, two enclosed squares

For the white fixed file, the following cases $(db, ds2)$ are possible:

- Case (0, 0): free file
- Case (1, 0): blocked file, no enclosed square
- Case (1, 1): blocked file, one enclosed square

Let $\Pi_w = \{(0, 0), (1, 0), (1, 1)\}$ and $\Pi_s = \{(0, 0), (1, 0), (1, 1), (1, 2)\}$. The files can be combined in any way, so Π is the set of all such combinations:

$$\Pi = \{(dw, db, ds1, ds2) \mid (dw, ds1) \in \Pi_s, (db, ds2) \in \Pi_w\}.$$



Fig. 17: The black fixed file is free. ($dw = ds1 = 0$)



Fig. 18: The black fixed file is blocked. No square is enclosed.
($dw = 1, ds1 = 0$)



Fig. 19: The black fixed file is blocked. One square is enclosed. ($dw = ds1 = 1$)



Fig. 20: The black fixed file is blocked. Two squares are enclosed. ($dw = 1, ds1 = 2$)



Fig. 21: The white fixed file is free. ($db = ds2 = 0$)



Fig. 22: The white fixed file is blocked. No square is enclosed.
($db = 1, ds2 = 0$)



Fig. 23: The white fixed file is blocked. One square is enclosed.
 $(db = 1, ds2 = 1)$

With this, we can also determine an upper bound on the number of possible pawn distributions when $fixp = 1$.

Theorem 4.12 *Let*

- Π be defined as above,
- $gopp = minopp - dw - db$,
- $x = (dw, ds1, db, ds2)$,

then $B_{1,wp,bp}$ is an upper bound on the number of ways to place wp non-fixed white pawns and bp non-fixed black pawns on the board when an en passant capture is possible, with

$$B_{1,wp,bp} = \sum_{x \in \Pi} \sum_{s=0}^{4 \cdot gopp} m(gopp, s, 6) \cdot \binom{44 - minopp - gopp - ds1 - ds2 - s}{wp - minopp + db, bp - minopp + dw}.$$

Proof We use x as shorthand for $(dw, ds1, db, ds2)$ so the formula fits on one line.

$$\sum_{s=0}^{4 \cdot gopp} m(gopp, s, 6) \cdot \binom{44 - minopp - gopp - ds1 - ds2 - s}{wp - minopp + db, bp - minopp + dw}$$

describes — similarly to Theorem 4.11 — the number of ways to distribute non-fixed pawns. In contrast to Theorem 4.11, some adjustments must be made due to the existence of fixed pawns.

As mentioned, these two fixed files are not treated as “normal” blocked files, so the minimum number of blocked files is now $gopp := minopp - dw - db$. Additionally, there are only six files left (namely those that are not fixed), from which the blocked files can be chosen. Therefore we use $m(gopp, s, 6)$ instead of $m(minopp, s, 8)$, and sum up to $s = 4 \cdot gopp$ instead of $4 \cdot minopp$. The multinomial coefficient describes the number of ways to place the remaining non-fixed pawns. Let’s see how we get the numbers $44 - minopp - gopp - ds1 - ds2 - s$, $wp - minopp + db$, and $bp - minopp + dw$. Already $minopp - db$ of the wp non-fixed white pawns have been placed, leaving $wp - minopp + db$ non-fixed white pawns to be placed. Analogously, $bp - minopp + dw$ black pawns must still be placed.

$$z := 2 + 2 + minopp + gopp + ds1 + ds2 + s$$

squares are no longer available. $2 \cdot gopp$ squares are already occupied by the pawns in the blocked files, and $dw + db$ more squares by non-fixed pawns in the fixed files. Note that $dw + db + 2 \cdot gopp = gopp + minopp$. If i is the black fixed file, with $i \in \{a, \dots, h\}$, then squares $i6$ and $i7$ are not available, because the black fixed pawn moved from $i7$ to $i5$ in the last move, and no pieces can be in the way. Two more squares are occupied by the two fixed pawns. Thus there are $48 - z = 44 - minopp - gopp - ds1 - ds2 - s$ squares left to position the rest of the pawns.

Finally, summing over all possible situations for the fixed files yields $B_{1,wp,bp}$ as an upper bound on the number of ways to place wp non-fixed white and bp non-fixed black pawns if an en passant capture is possible. $\square$

In the code, these upper bounds are computed in the second part of the function `pawnsfunction` (lines 114-138) by calling the function `pawnsep` for each combination $(dw, db, ds1, ds2)$. The function `pawnsep` calculates

$$\sum_{s=0}^{4 \cdot gopp} m(gopp, s, 6) \cdot \binom{44 - minopp - gopp - ds1 - ds2 - s}{wp - minopp + db, bp - minopp + dw}.$$

After placing the pawns, there remain

$$vF := 64 - 4 \cdot fixp - fixwk - fixbk - fwr - fbr - wp - bp$$

squares (line 219 in the code) for the other pieces. The factor $4 \cdot fixp$ is used because four squares are unavailable if there are fixed pawns: the squares of the two fixed pawns themselves plus the two squares over which the black fixed pawn traveled in its last move. The distribution of the remaining pieces is computed by a multinomial coefficient (line 221). Here, the products of factorials `wprod` and `bprod` computed in lines 174 and 197 are used to save runtime.

Tromp obtains an upper bound of about $8,7 \cdot 10^{45}$ as the result of his computations. This is an improvement of 50.9% over Chinchalkar’s upper bound. The partial results for different values of fwr , fbr , and $fixp$ are shown in the

following figure. Note that these values have not yet been multiplied by two and thus only apply to the side White to move.

```
[Running] cd "/Users/christianbuchs/Documents/Studium/Bachelorarbeit/C Project/Folder/" && gcc direktekopie
Stadium/Bachelorarbeit/C Project/Folder/"direktekopie
fwr = 0 fbr = 0 fixp = 0 gerundet: 4.317117e+45 exakt: 4317116501822869085736524809553156518216466432
fwr = 0 fbr = 0 fixp = 1 gerundet: 3.199960e+43 exakt: 31999595200554360726978953532556683563237376
fwr = 0 fbr = 1 fixp = 0 gerundet: 6.922143e+42 exakt: 6922142764258201498990574886765035111055360
fwr = 0 fbr = 1 fixp = 1 gerundet: 5.444438e+40 exakt: 54444384271358764337920637881472010682368
fwr = 0 fbr = 2 fixp = 0 gerundet: 1.365308e+41 exakt: 136530769983397185483934838882727423377408
fwr = 0 fbr = 2 fixp = 1 gerundet: 1.035366e+39 exakt: 1035365880137246104468971868036265934848
fwr = 1 fbr = 0 fixp = 0 gerundet: 6.922143e+42 exakt: 6922142764256625601320564597675088525918208
fwr = 1 fbr = 0 fixp = 1 gerundet: 5.430835e+40 exakt: 54308353406843120725028311050169928908800
fwr = 1 fbr = 1 fixp = 0 gerundet: 1.174542e+40 exakt: 11745419798220500118058263406702695022592
fwr = 1 fbr = 1 fixp = 1 gerundet: 9.817252e+37 exakt: 98172517157462428199580525005092421632
fwr = 1 fbr = 2 fixp = 0 gerundet: 2.359583e+38 exakt: 235958281120053141339249377081458425856
fwr = 1 fbr = 2 fixp = 1 gerundet: 1.903337e+36 exakt: 1903336650820530413600515787306565632
fwr = 2 fbr = 0 fixp = 0 gerundet: 1.365308e+41 exakt: 136530769983304939608194964218180643323904
fwr = 2 fbr = 0 fixp = 1 gerundet: 1.028638e+39 exakt: 1028637537647016138320248178990487961600
fwr = 2 fbr = 1 fixp = 0 gerundet: 2.359583e+38 exakt: 2359582811199212550808115793629930323968
fwr = 2 fbr = 1 fixp = 1 gerundet: 1.895643e+36 exakt: 1895642836533361711037586679714021376
fwr = 2 fbr = 2 fixp = 0 gerundet: 4.729971e+36 exakt: 4729971278279345018382308284936749056
fwr = 2 fbr = 2 fixp = 1 gerundet: 3.663529e+34 exakt: 36635290891907343302044348405776384
Totale Summe: 8726713169814947753088220017782443029005598720
[Done] exited with code=1 in 9.248 seconds
```

Fig. 24: Output of the C-translation of Tromp's program.

5 My Extension

My extension is based on the C-translation of Tromp’s code. Thus, only my modifications to the program need to be proven here.

A chess position is illegal if one side is in check, yet it is not that side’s turn. In my extension, I will filter out some such illegal chess positions. I will, again, assume White to move. That means that every chess position in which the black king is in check is illegal. To generally check whether the black king is in check, we need:

- the black king’s location,
- the location of the checking piece,
- which piece it is (since a knight checks differently from a bishop),
- whether there are any pieces blocking that check.

Handling all of this requires a very large number of case distinctions, which would exceed the scope of this thesis. Therefore, in this chapter, I will look at a more specific case that requires far fewer distinctions. At the end of the chapter, I will outline how a program could be implemented that filters out many more illegal chess positions. All parts of this chapter are my own work.

Since the improvement in Tromp’s upper bound over Chinchalkar’s primarily comes from the clever calculation of pawn distributions, we also want to use that advantage. Therefore, we place pawns on the board first, and only then place the black king. Because Tromp’s functions only calculate how many pawn distributions exist, we do not know exactly where the pawns stand when we place the black king. To figure this out, we would again need a very large number of case distinctions. Thus we will mostly limit ourselves to squares outside the pawn territory.

We distinguish four different ways of placing the black king, which we call $kpos \in \{0, 1, 2, 3\}$:

- Case $kpos = 0$: The king is on one of the four corners (a1, h1, a8, h8).
- Case $kpos = 1$: The king is on rank one or rank eight, but not in a corner (b1-g1, b8-g8).
- Case $kpos = 2$: The king is in the pawn territory.
- Case $kpos = 3$: The king is fixed (he stands on e8).

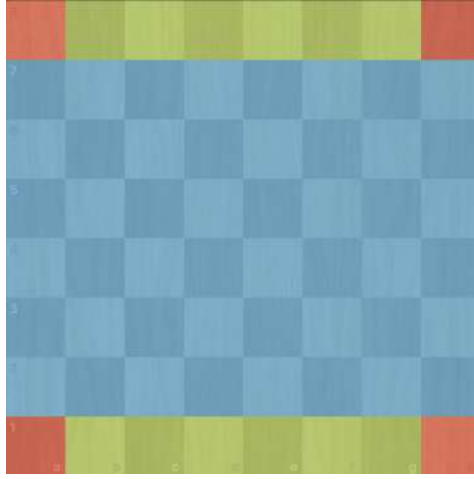


Fig. 25: Possible positions of a non-fixed king. Markings: $kpos = 0$: red, $kpos = 1$: green, $kpos = 2$: blue.



Fig. 26: Possible positions of a fixed king. $kpos = 3$: orange.

We need to know how many ways $kmult$ there are to place the black king in each of these $kpos$ cases.

Lemma 5.1 *It holds that:*

$$kmult = \begin{cases} 4 - fwr & \text{if } kpos = 0 \\ 12 - 3 \cdot fixwk - fwr & \text{if } kpos = 1 \\ k & \text{if } kpos = 2 \\ 1 & \text{if } kpos = 3 \end{cases}$$

with

$$k = 48 - 4 \cdot fixp - wp - bp - 3 \cdot fixwk - fwr.$$

Proof We prove each case:

Case $kpos = 0$: There are four corners. Only fixed white rooks can already occupy corners, so $kmult = 4 - fwr$.

Case $kpos = 1$: There are six squares in rank 1 and six in rank 8 that are not corners, giving twelve total squares. Only a fixed white king can occupy one of those squares (e1) at this point. Since kings cannot stand next to each other, the black king cannot stand on d1 or f1 in that case. Moreover, the black king cannot be on squares adjacent to any fixed white rook (b1, g1). Hence for each fixed white rook, one less square is available. Thus $kmult = 12 - 3 \cdot fixwk - fwr$.

Case $kpos = 2$: Ranks two through seven have 48 squares. Of these, $wp + bp$ are occupied by white and black non-fixed pawns. If $fixp = 1$, four more squares are unavailable: the squares of the two fixed pawns plus the two squares the black fixed pawn jumped over. If there is a fixed white king ($fixwk = 1$), squares d2, e2, and f2 cannot be occupied by the black king, as kings cannot stand next to each other. The black king also cannot stand on squares adjacent to fixed white rooks (a2, h2). Hence for each fixed white rook, one fewer square is available. Thus in total we have $kmult = 48 - 4 \cdot fixp - wp - bp - 3 \cdot fixwk - fwr$ squares left.

Case $kpos = 3$: If the king is fixed, he stands on e8, so $kmult = 1$. □



Fig. 27: Squares that may be unavailable for $kpos$ due to fixed pieces. The fixed pawns may also be in other files. Markings: $kpos = 0$: red, $kpos = 1$: orange, $kpos = 2$: blue.

Next, we want to place the checking pieces. Since it would be very involved to find all squares from which a piece can give check and how many ways to block such a check, we will only look at situations where it is certain that the checking piece can stand right next to the black king, and thus blocking is impossible. Moreover, we want that checking piece not to be in the pawn territory, just like the black king, so we do not have to worry whether a pawn is already on that square. Because knights and bishops would have to stand in the pawn territory to check a king not in the pawn territory, we only focus on queens and rooks. The variable $qspaces$ tells us how many ways a queen or rook can check the king from a neighboring square outside the pawn territory.

Lemma 5.2 *Let $qspaces$ be the number of ways to place a queen or rook on a neighboring square such that neither the checking piece nor the king is in the pawn territory. Then:*

$$qspaces = \begin{cases} 1 & \text{if } kpos = 0 \\ 2 & \text{if } kpos \in \{1, 3\} \\ 0 & \text{if } kpos = 2. \end{cases}$$

Proof Because both the rook or queen and the king must be in the same rank outside the pawn territory, we only need to count the neighbor squares in that rank. If the rank is in the pawn territory ($kpos = 2$), then $qspaces = 0$ by

definition. If the king is in a corner ($kpos = 0$), there is only one neighbor square in the same rank. If $kpos \in \{1, 3\}$, there are two such squares. $\square$

The various cases are illustrated in the following figures.



Fig. 28: If the king is in a corner ($kpos = 0$), there is only one possible square for the rook or queen.



Fig. 29: If the king is on rank 1 or rank 8 but not in a corner ($kpos \in \{1, 3\}$), there are two possible squares for the rook or queen.



Fig. 30: If the king is in the pawn territory ($kpos = 2$), it is not guaranteed that squares for a rook or queen are free, because the situation shown here could occur. Therefore, we conservatively set $qspaces = 0$.

Because chess positions in which the black king is in check are illegal, there can be no white queens or white rooks on the $qspaces$ neighboring squares of the black king.

Theorem 5.3 *Let*

- $qspaces$ be defined as in Lemma 5.2,
- $kmult$ be defined as in Lemma 5.1,
- $space := 64 - 4 \cdot fixp - fixwk - fwr - fbr - wp - bp$.

Then K is an upper bound on the number of ways of placing the black king on a square of the current $kpos$ -case and wD white queens and wr white non-fixed rooks on the board, namely:

$$K = kmult \cdot \binom{space - 1 - qspaces}{wD, wr}.$$

Proof $kmult$ is the number of ways to place the black king in the given $kpos$ -case. The variable $space$ is the number of available squares after the pawns and all fixed pieces except the black king have been placed. The multinomial coefficient counts the ways to place wD white queens and wr non-fixed white rooks. The “-1” in $space - 1 - qspaces$ accounts for the black king, and $qspaces$ squares are disallowed for white queens and rooks because they would put the black king in immediate check. $\square$

The multiplier $multFWK$ is an upper bound on the number of ways to place the white king.

Lemma 5.4 *Let $space$ be as defined in Theorem 5.3. Then $multFWK$ is an upper bound on the number of ways to place the white king, where*

$$multFWK = \begin{cases} 1 & \text{if } fwr > 0 \\ space - 1 - wr - wD & \text{if } fwr = 0. \end{cases}$$

Proof If $fwr > 0$, the white king is fixed, and there is only one way to place it. If $fwr = 0$, the white king is placed on the board after the pawns, black king, white queens, white rooks, and all fixed pieces have been placed. Since $space$ is the number of squares available before placing the black king, wD white queens, and wr white rooks, after placing them there are at most $space - 1 - wr - wD$ squares left for the white king. $\square$

Let us consider the differences between my code and Tromp's C-translation. All line references refer to my code.

Looking at the new functions in my code:

`multinom` (lines 163-175) computes multinomial coefficients of the form $\binom{n}{i,j}$ and stores them in the array `mn`. `binomial` (lines 149-160) is just a helper function computing binomial coefficients, stored in the array `binom`. `multiplebinomrook` (lines 178-196) computes the upper bound K from Theorem 5.3 and stores it in the array `mb`.

Turning to the modified existing functions:

In the function `count`, line 222 checks if $fbr = 0$, i.e. if there is no fixed black king. If so, it goes through $kpos = 0, 1, 2$; otherwise $kpos = 3$. If there is no fixed black king, lines 262-266 compute $kmult$ (Lemma 5.1). If there is a fixed black king, line 289 sets $kmult = 1$. The variable $space$ in lines 259 and 326 differs from Tromp's `space` because it does not subtract $fixbk$ yet, as we only place the black king (fixed or not) in `multiplebinomrook`.

Whereas Tromp's code uses a multinomial coefficient to distribute all pieces after placing the pawns, my code uses the `mb` array's value for placing the black king and the wD white queens and wr white rooks, the multiplier `multFWK` for placing the white king, and finally uses multinomial coefficients via `mn` to distribute the other pieces.

My program's final result is

$$8.374.836.394.172.425.477.842.765.437.670.291.687.010.729.984 \approx 8,73 \cdot 10^{45}.$$

Hence, my upper bound is about 4.0% smaller than Tromp's. The partial results for fwr , fbr , and $fixp$ are shown in the next figure. Note that these are still unmultiplied by two and thus only apply to White to move.

```

[Running] cd "/Users/christianbuchs/Documents/Studium/Bachelorarbeit/C Project/Folder/" && gcc arbeitsbereich
Bachelorarbeit/C Project/Folder/"arbeitsbereich
fwr = 0 fbr = 0 fixp = 0 gerundet: 4.144773e+45 exakt: 4144773441728498560365341708306211071907069952
fwr = 0 fbr = 0 fixp = 1 gerundet: 3.066606e+43 exakt: 30666062795919153678752107982269681865064448
fwr = 0 fbr = 1 fixp = 0 gerundet: 5.737524e+42 exakt: 5737523849016796789065681742333186450391040
fwr = 0 fbr = 1 fixp = 1 gerundet: 4.532929e+40 exakt: 45329287090512824733318495978669733838848
fwr = 0 fbr = 2 fixp = 0 gerundet: 1.131520e+41 exakt: 113151969932855792273952879950937534234624
fwr = 0 fbr = 2 fixp = 1 gerundet: 8.620150e+38 exakt: 862015042677659888435339220996069523456
fwr = 1 fbr = 0 fixp = 0 gerundet: 5.912294e+42 exakt: 5912293887285700630466356389189630356881408
fwr = 1 fbr = 0 fixp = 1 gerundet: 4.588557e+40 exakt: 45885574713994993037604035509438063312896
fwr = 1 fbr = 1 fixp = 0 gerundet: 9.925824e+39 exakt: 9925823765983077419511628091318580805632
fwr = 1 fbr = 1 fixp = 1 gerundet: 8.352274e+37 exakt: 83522735541271808441166513994507223040
fwr = 1 fbr = 2 fixp = 0 gerundet: 1.994760e+38 exakt: 199476036685788573268802054854874759168
fwr = 1 fbr = 2 fixp = 1 gerundet: 1.620145e+36 exakt: 1620144867787475934843231477021278208
fwr = 2 fbr = 0 fixp = 0 gerundet: 1.123917e+41 exakt: 112391745590857961575565702234681938804736
fwr = 2 fbr = 0 fixp = 1 gerundet: 8.350204e+38 exakt: 835020432733113420235106718494434000096
fwr = 2 fbr = 1 fixp = 0 gerundet: 2.030196e+38 exakt: 203019648536401711584970237088844742656
fwr = 2 fbr = 1 fixp = 1 gerundet: 1.645616e+36 exakt: 1645615687922815973514991398727712768
fwr = 2 fbr = 2 fixp = 0 gerundet: 4.072906e+36 exakt: 4072906148386374497746531610594902016
fwr = 2 fbr = 2 fixp = 1 gerundet: 3.183536e+34 exakt: 31835362284437777765927429890637824
Totale Summe: 8374836394172425477842765437670291687010729984
Verbesserung um 4.0 Prozent
[Done] exited with code=1 in 21.318 seconds

```

Fig. 31: Output of my program.

The runtime of the program more than doubled through my extension (from 9.2 to 21.3 seconds). Although it is possible to improve the upper bound further, it is likely impossible to do so without a strong increase in runtime, because even more case distinctions would be needed to filter out an increasingly small number of illegal chess positions.

One could further improve the program by extending `pawnsfunction` so that for specific squares, it calculates how many pawn distributions do not place a pawn on that square. Using this number, one could distinguish between squares that are definitely free and squares that might be blocked. One might then place the black king in more complicated checking scenarios, e.g., by knights or bishops, and thus filter out more illegal chess positions. If it were feasible runtime-wise to compute how many pawn distributions leave certain combinations of squares unoccupied, that could be used to handle positions where the black king is in the pawn territory or where the checking piece is separated from the black king by one or more squares.

Personally, I do not expect this upper bound to improve by more than about 30% in the coming years, since the workload to implement new filters grows rapidly while the yield in filtering out illegal positions shrinks. Improvements of about 10% up to at most 20% seem plausible to me.

6 Estimation via a Confidence Interval

A large part of Tromp’s work deals with computing a confidence interval Ω at a 95% confidence level in which the number L of legal chess positions lies. Tromp carries out this procedure for several sample sizes n . Among other things, he draws two samples of size $n = 1,000,000$ each and then combines them into one sample of size $n = 2,000,000$. We will first explain how the sample was implemented, then discuss the mathematical background, and finally look at Tromp’s findings for $n = 100$, $n = 10,000$, and for $n = 1,000,000$ and $n = 2,000,000$. All concepts are Tromp’s, but I have prepared and explained them more clearly, because his documentation is often quite terse and leaves a lot to the reader.

6.1 Implementation of the Sample

In this subsection, the definition of ranking, the definition of multiplicities, the explanation of the modified FEN notation, the pseudo-random number generation, and Tromp’s procedure for generating legal “urpositions” for invertibility tests come from him. The rest is my own contribution.

Tromp implements a sample of size n by generating n pseudo-random numbers in Haskell and converting them into “urpositions.” Later, we will explain what urpositions are. The conversion from pseudo-random numbers to urpositions and back from urpositions to pseudo-random numbers is done by an invertible ranking.

Definition 6.1 (Ranking) *A ranking R for a set G is a tuple $(size, u, r, c)$ with*

- *the size of G : $|G| = size \in \mathbb{N}$,*
- *an unrank function $u : \{0, \dots, size - 1\} \rightarrow G$,*
- *a rank function $r : G \rightarrow \{0, \dots, size - 1\}$,*
- *a comparison function c that compares pairs of numbers (ranked objects).*

A ranking is invertible if r is injective.

Remark At the start of his documentation, Tromp defines a ranking only as a triple consisting of *size*, a rank function, and an unrank function. He later extends this definition by adding the comparison function c so that he can sort the rankings and hence better test invertibility.

Invertibility is important so that two distinct urpositions are never mapped to

the same number.

It does not make sense to choose all chess positions as the population for the sample, since by [3], there are $13^{64} \cdot 2 \cdot 9 \cdot 2^4 = 5,6 \cdot 10^{73}$ chess positions. Thus there are at least $6,7 \cdot 10^{27}$ times more illegal than legal positions, and an immense sample size would be needed to get meaningful results. Modern computers do not have enough computing power to check the legality of so many positions.

Let U be the set of all positions counted in the upper bound from Chapter 4. We intentionally do not say “chess positions” here, because some positions were counted multiple times in the upper bound, as we will soon see. The set U is much more suitable as a population, because it has only about $8,7 \cdot 10^{45}$ elements. That is why Tromp introduces urpositions (short for “uniquely rankable positions”), whose population is exactly U . Urpositions are chess positions with additional information so that each of them is counted exactly once in the upper bound from Chapter 4. Hence U is the set of all urpositions, and the variable *size* in the ranking is the upper bound computed in Chapter 4.

Earlier, we mentioned that some chess positions are counted multiple times in the upper bound. In other words, there are chess positions that belong to multiple urpositions. As an example, consider the chess position that is determined by $fixbr = 1$, $fixwr = 1$, “the pawn on d5 can be captured en passant,” “White to move,” and the piece placement from the following figure. One of the additional pieces of information for a urposition is which squares the fixed pawns occupy. There are multiple urpositions for this one chess position, because either the white pawn on c5 or the white pawn on e5 could be the “fixed white pawn.” In the code as well, both urpositions are counted individually by the multiplier *multEP*, although they describe the same position. The number of urpositions describing the same chess position p is the “multiplicity” of p , written $m(p)$. A urposition has multiplicity m if the corresponding chess position has multiplicity m .



Fig. 32: This piece distribution belongs to a chess position described by more than one urposition.

Because a key part of sampling is the legality test, the urpositions must be input and output efficiently. Tromp uses the FEN notation with the following modifications:

- The en passant square in the FEN is only used if there is actually a pawn to be captured en passant (blue).
- Two squares are appended at the end, describing the multiplicity of the chess position (teal) and that urposition's index in a numbering of all urpositions describing the same position (orange).

This is an example of the modified FEN notation:

```
r3kb1r/ppp1pp1p/3p4/5PpP/8/8/PPPPP1P1/RNBQKBNR w KQkq !g6| 99|
140|
```

These modified FEN notations are used for input to the legality checking programs. Note that for checking the legality of a urposition, these extra bits of information are irrelevant. Only the usual FEN contents matter. So a urposition is legal if and only if the corresponding chess position is legal.

Tromp tests whether his ranking is invertible by generating legal urpositions, ranking them, then unranking them, sorting them, and checking that the set of urpositions before ranking/unranking is the same as after. For generating legal urpositions, he let a program play chess by choosing a random move out of the set of all possible moves after each move, and stopping with a 1% probability after every move. He then sorted the results with the comparison function.

This test suggested that his ranking is invertible. He also uses sampling to verify invertibility again.

For the actual sampling, Tromp uses pseudo-random numbers from Haskell's random package. The legality test uses various proof-of-legality programs. Especially for the bigger samples, some urpositions remained that these programs could not classify as legal or illegal, so they had to be checked manually.

6.2 Mathematical Background

In this section, we look at Tromp's proof that he uses an unbiased estimator, and see that $Y(S)$ is approximately normally distributed. We add some detail to his normal distribution argument. For this section:

- I_m is the number of legal chess positions with multiplicity m .
- n_m is the number of urpositions with multiplicity m .
- $L = \sum_{m \in \mathbb{N}} I_m$ is the total number of legal chess positions.
- LU is the set of all legal urpositions.
- $N = \sum_{m \in \mathbb{N}} n_m$ is the total number of urpositions.
- If $X \in U$ and m is the multiplicity of X , define the random variable $Y(X)$ by

$$Y(X) = \begin{cases} 0 & \text{if } X \text{ is illegal,} \\ \frac{N}{m} & \text{if } X \text{ is legal.} \end{cases}$$

Theorem 6.2 Y is an unbiased estimator for L .

Proof We prove this with the following equation:

$$\mathbb{E}[Y] = \sum_{X \in U} P(X \in LU \wedge M(X) = m) \cdot Y(X) = \sum_{m \in \mathbb{N}} \frac{m \cdot I_m}{N} \cdot \frac{N}{m} = \sum_{m \in \mathbb{N}} I_m = L.$$

The first equality follows from the definition of expectation. For the second equality, note that there are $m \cdot I_m$ urpositions with multiplicity m that are legal, and we are choosing from N total urpositions, so the probability is $\frac{m \cdot I_m}{N}$. $\square$

For a sample S of size n with n_s legal urpositions, Tromp estimates L by averaging the individual estimates:

$$Y(S) = \frac{1}{n} \sum_{u \in S \cap LU} \frac{N}{m(u)} = \frac{n_s}{n} \cdot N \cdot \frac{1}{n_s} \sum_{u \in S \cap LU} \frac{1}{m(u)}. \quad (1)$$

Since every legal chess position is legal for each of its urpositions, a position with more urpositions has a higher chance of appearing in the sample, so the

raw fraction of legal urpositions is not enough; we also need the multiplicity. This can be seen in formula (3) from the factor

$$\frac{1}{n_s} \sum_{u \in S \cap LU} \frac{1}{m(u)},$$

the inverse average multiplicity (*IDM*) of the legal urpositions. The smaller their multiplicity is, the bigger the *IDM* and the bigger $Y(S)$, since fewer urpositions correspond to each legal position but the same number of legal urpositions is found. The next theorem shows that $Y(S)$ is approximately normally distributed.

Theorem 6.3 *Let $Y(S)$ be defined as in (3). Then $Y(S)$ is approximately normally distributed.*

Proof We already know $\mathbb{E}[Y] = L < \infty$. As $1 \leq Y \leq N$ and $1 \leq L \leq N$, we have $\text{Var}(Y) = \mathbb{E}((Y - L)^2) < \infty$. Let the sample S consist of n urpositions $(X_1, \dots, X_n)$. Clearly, the $Y(X_i)$ are i.i.d., and since their expectation and variance are finite, the Central Limit Theorem implies $Y(S)$ is approximately normally distributed. $\square$

This approximate normal distribution helps Tromp compute confidence intervals, done in *estimate.pl*.

6.3 Sample Results

All results come from Tromp unless otherwise stated. The analyses are mine.

In the small sample $S1$ with $n = 100$, there are $n_{S1} = 3$ legal urpositions, each with multiplicity 1, so $IDM = 1$. This gives Tromp a 95% confidence interval for L with bounds $(2.6 \pm 2.9) \cdot 10^{44}$. Clearly, all upper bounds are still way larger.

In the medium sample $S2$ with $n = 10,000$, $n_{S2} = 538$ urpositions are legal, and $IDM \approx 0.97847$. The 95% confidence interval has bounds $(4.59 \pm 0.38) \cdot 10^{44}$. Interestingly, the relative frequency of legal urpositions here is 5.38%, about twice that of the small sample $S1$ (3.00%). This suggests $S1$ underrepresented legal urpositions. The smaller sample partially compensates for that by having a larger *IDM*.

Tromp drew two samples of size $n = 1,000,000$. The first sample, $S3$, was made with version 1.1 of the Haskell random package; the second, $S4$, with version 1.2. Finally, he combined both into a sample $S5$ of size $n = 2,000,000$. We look more closely at $S3$ and only list final results for $S4$ and $S5$.

The first analysis of $S3$ gave:

```

1 $ src/legal < sortedRndimFENs | cut -f2 -d '$\t' | sort | uniq -c |
   sort -rn
2 492045 Illegal Both Kings in Check
3 173401 Illegal Side not to move in Check
4 102541 Illegal Adjacent Kings
5 79019 Illegal Bishops Too Monochromatic
6 53063 Single Check
7 35584 No Checks
8 29962 Illegal Triple Check
9 27415 Illegal Double Check
10 6256 Discovered Double Check
11 714 Illegal Double Knight Check

```

Only “Single Check,” “No Check,” or “Discovered Double Check” can be legal. This filters 94,903 urpositions, which were then fed to Peter Österlund’s Texel program. That left 65 to check manually.

S_3 has $n_{S_3} = 56,011$ legal urpositions with $IDM \approx 0.98013$. The 95% confidence interval is $(4.79082 \pm 0.0387698) \cdot 10^{44}$. These numbers are close to S_2 except that S_3 ’s confidence interval is about ten times tighter, as we would expect with 100 times the sample size.

Fewer details are published about S_4 , but from S_3 and S_5 , I infer $n_{S_4} = 56,743$ and $IDM \approx 0.98006$, giving a 95% confidence interval $(4.85304 \pm 0.039004) \cdot 10^{44}$. The values for S_3 and S_4 are nearly identical.

Combining S_3 and S_4 into S_5 yields $n_{S_5} = 112,754$ legal urpositions with $IDM \approx 0.98010$ and a 95% confidence interval $(4.82193 \pm 0.0274973) \cdot 10^{44}$. Merging the two million-samples has thus shrunk the margin of this confidence interval by about 30%.

Here is the *estimate.pl* output:

```

1 $ cat Texel.out.94903 Texel.out.95544 | src/estimate.pl 2000000
2 Exp[Y] ~ 0.0552548 * N
3 Var[Y] ~ 0.051689 * N^2
4 Sigma[Y] ~ 0.227352 * N
5 Sigma[Y[S]] ~ 0.000160762 * N
6 n = 2000000 samples 112754/190447 legal 108547 3418 705 56 3 23
   2 avg 1/mult 0.980095015106633
7 estimate 4.82193e+44 +- 2.74973e+42 at 95% confidence

```

Hence, the number of legal chess positions L is about $4.82 \cdot 10^{44}$ and with 95% probability lies in $(4.82 \pm 0.03) \cdot 10^{44}$. The upper bound $8.73 \cdot 10^{45}$ is almost exactly 18 times the top of that confidence interval.

7 Conclusion

Through the upper bound $8.73 \cdot 10^{45}$ and the confidence interval $(4.82 \pm 0.03) \cdot 10^{44}$, we can already predict quite precisely the number of legal chess positions. This is especially remarkable given that the total number of chess positions is $5.6 \cdot 10^{73}$ by [3].

We can expect further refinement of both the upper bound and the confidence interval in the future, but how big these improvements will be remains open, since there are limiting factors for both the bounding computations and sample evaluation.

Right now, the main limiting factor for the sample evaluation is coverage by legality-checking programs. In sample *S3*, 65 of the 1,000,000 urpositions had to be checked manually. Assuming the proportion of manual checks is similar for all samples, a sample of size 10^7 would require 650 manual checks, and size 10^9 would require 65,000. Hence significant improvement in the confidence interval precision is only possible if the legality-checking programs can cover far more urpositions automatically.

For computing upper bounds, both the effort-to-yield ratio and computer power are limiting. One would need more and more detailed analyses of which pieces stand on which squares how often, requiring many case splits. This lengthens runtime considerably. And as the number of illegal positions that remain shrinks, it gets ever harder to filter them out. Without a new approach, I consider it unlikely my upper bound will improve by more than 30% in the next few years.

We should still give praise to Shannon's estimate of 10^{43} , which, despite the simplicity of its calculation, is only off by a factor of about 48.

In conclusion, we see that computers do enable us to find a very good estimate of the number of legal chess positions, but likely never to compute the exact number — just as today's computers play chess better than the best grandmasters but will never play perfectly.

But that is exactly what is fascinating, both in mathematics and in chess.

8 References

- [1] Chinchalkar, Shirish: An upper bound for the number of reachable positions, ICGA Journal, Vol. 19, No. 3, pp. 181–183, 1996 (3 pages)
- [2] Entriken, William: Github Post chess-upper-bound-armies, version from 08/07/2016,
<https://github.com/fulldecent/chess-upper-bound-armies>,
accessed 09/09/2023 13:47 CEST
- [3] Österlund, Peter: Github Post texel, version from 09/25/2023,
<https://github.com/peterosterlund2/txel/blob/master/doc/proofgame.md>,
accessed 09/28/2023 13:57 CEST
- [4] Shannon, Claude E.: XXII. Programming a Computer for Playing Chess, Philosophical Magazine, Ser. 7, Vol. 41, No. 314, 1950 (18 pages)
- [5] Tromp, John: Github Post ChessPositionRanking, version from 05/12/2022,
<https://github.com/tromp/ChessPositionRanking>,
accessed 09/09/2023 18:22 CEST

Image sources: All figures used are my own screenshots. The chess positions were created with the website <https://www.chess.com>.

9 Appendix

The appendix contains the following program codes:

- The C-translation of Tromp's Haskell code *kopievtromp.c*
- My extension *meincode.c*

kopievontromp.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  extern double pow(double, double);
6
7  #define min(a,b) ((a) < (b) ? (a) : (b))
8  #define max(a,b) ((a) > (b) ? (a) : (b))
9
10
11 double fac[65];          // fac[n] = n!
12 double choose2 [49][9][9]; // choose[n][j][k] = Multinomialkoeffizient  $n! / (j! * k! * (n - j - k)!)$ 
13 double mopps_[9][33][9]; // Umsetzung von m(p,s,n) aus Satz 4.10
14 double pawns[2][9][9][9];
15
16
17 //Berechnet die Werte von fac[i]
18 void factorial() {
19     int i, j;
20     for (i=0; i<65; i++) {
21         fac[i] = 1.0;
22         for (j=2; j<=i; j++) {
23             fac[i] *= j;
24         }
25     }
26 }
27
28 // Berechnet die Werte von choose2[n][k1][k2]
29 void choose2function(){
30     int n, k1, k2;
31     for (n = 0; n <= 48; n++){
32         for (k1 = 0; k1 <= 8; k1++){
33             for (k2 = 0; k2 <= 8; k2++){
34                 if (n >= k1 + k2){
35                     choose2[n][k1][k2] = fac[n] / (fac[k1] * fac[k2] * fac[n - k1 - k2]);
36                 } else {
37                     choose2[n][k1][k2] = 0;
38                 }
39             }
40         }
41     }
42 }
43
44 //Berechnet die Werte von mopps[p][s][n]
45 double opps(int p, int s, int n) {
46     double res = 0.0;
47     if (p == 0 && s == 0) {
48         res = 1;
49     } else if (p == 0 || n == 0) {
50         res = 0;
51     } else {
52         res = mopps_[p][s][n - 1];
53         for (int i = 0; i <= (s < 4 ? s : 4); ++i) {
54             res += (5 - i) * mopps_[p - 1][s - i][n - 1];
55         }
56     }
57     return res;
58 }
59
60 //Weißt mopps[p][s][n], die in opps(p,s,n) berechneten Werte zu
61 void moppsfunction(){
62     double max = 0.0;
63     for (int p = 0; p <= 8; ++p) {
64         for (int s = 0; s <= 32; ++s) {
65             for (int n = 0; n <= 8; ++n) {
66                 mopps_[p][s][n] = opps(p, s, n);
67             }
68         }
69     }
70 }
71

```

```

72 // Berechnet die innere Summe in Satz 4.12
73 // dw = 1 <=> weißer Bauer in schwarzer fixierter Spalte
74 // db = 1 <=> schwarzer Bauer in weißer fixierter Spalte
75 // ds = ds1 + ds2 = Anzahl an eingeschlossenen Feldern in fixierten Spalten
76 double pawnsep(int dw, int db, int ds, int wp, int bp, int opp){
77     int opp_ = opp - dw - db;           // Mindestanzahl blockierter nicht-fixierter Spalten
78     int s;                             // Anzahl eingeschlossener Felder
79     double res = 0.0;
80     for (s = 0; s <= 4 * opp_; s++){
81         if(bp + dw >= opp && wp + db >= opp){ // Damit keine negative (also nichtdefinierte) Komponente von choose2 aufgerufen wird
82             res += mopps_[opp_][s][6] * choose2[44 - opp - opp_ - ds - s][wp + db - opp][bp + dw - opp];
83         }
84     }
85     return res;
86 }
87
88
89 void pawnsfunction(){
90     int s, wp, bp, opp;
91     int ds1, ds2;
92     double res;
93
94     // Fall: Kein Enpassant -> fixp = 0
95     // Berechnet obere Schranke aus Satz 4.11
96     for (wp = 0; wp <= 8; wp++){
97         for (bp = 0; bp <= 8; bp++){
98             for (opp = 0; opp <= 8; opp++){
99                 if (opp > min(wp, bp)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
100                     pawns[0][wp][bp][opp] = 0;
101                 } else {
102                     res = 0.0;
103                     for (s = 0; s <= 4 * opp; s++){
104                         res += mopps_[opp][s][8] * choose2[48 - 2 * opp - s][wp - opp][bp - opp];
105                     }
106                     pawns[0][wp][bp][opp] = res;
107                 }
108             }
109         }
110     }
111
112     // Fall: Enpassant -> fixp = 1
113     // Berechnet obere Schranke aus Satz 4.12
114     for (wp = 0; wp <= 8; wp++){
115         for (bp = 0; bp <= 8; bp++){
116             for (opp = 0; opp <= 8; opp++){
117                 res = 0;
118                 if (opp > min(wp + 1, bp + 1)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
119                     pawns[1][wp][bp][opp] = 0;
120                 } else {
121                     res = pawnsep(0, 0, 0, wp, bp, opp);
122                     for (ds1 = 0; ds1 <= 2 && opp > 1; ds1++){
123                         for (ds2 = 0; ds2 <= 1; ds2++){
124                             res += pawnsep(1, 1, ds1 + ds2, wp, bp, opp);
125                         }
126                     }
127                     for (ds1 = 0; ds1 <= 2 && opp > 0; ds1++){
128                         res += pawnsep(1, 0, ds1, wp, bp, opp);
129                     }
130                     for (ds2 = 0; ds2 <= 1 && opp > 0; ds2++){
131                         res += pawnsep(0, 1, ds2, wp, bp, opp);
132                     }
133                     pawns[1][wp][bp][opp] = res;
134                 }
135             }
136         }
137     }
138 }
139
140 // Berechnet eine obere Schranke (bis auf Multiplikatoren) für die Anzahl an Schachpositionen, bei denen Weiß am Zug ist
141 // in Abhängigkeit von fwr, fbr und fixp
142 double count(int fwr, int fbr, int fixp){
143     double summe = 0.0;
144     double h = 0.0; // Hilfsvariable, damit der Code nicht zu weit nach rechts geht & gedruckt werden kann
145     int np = 8 - fixp; // Maximal mögliche nicht-fixierte Bauern pro Seite
146     int fixwk = (fwr != 0 ? 1 : 0); // Es existiert ein fixierter König <=> Mind. 1 Fixierter Turm existiert
147     int fixbk = (fbr != 0 ? 1 : 0);

```

```

148 int uwr, wk, wq, wnp1, wr, wb, wnp2, wnp3, wn, wnp4, wproms, wp, wpx;
149 int ubr, bk, bq, bnp1, br, bb, bnp2, bnp3, bn, bnp4, bproms, bp, bpx;
150 int wpcs, bpcs, caps, maxuwp, maxubp, minopp, space;
151
152 // wprod und bprod sind Produkte von Fakultäten und dienen zur Berechnung eines Multinomialkoeffizienten
153 double wprod, bprod;
154
155 // Anzahl der weißen Figuren
156 uwr = 2 - fwr; // Da es 2 Türme zum Start gibt, müssen mind. 2 - fwr Türme nicht-fixiert sein.
157 wk = floor(uwr / 2); // Anzahl nicht-fixierter Könige
158
159
160 // Die Verringerung der Variable np zu wnp1 zu Variablen wie wnp4 hilft Figurenkombinationen herauszufiltern,
161 // für die zu viele Promotionen nötig wären.
162
163 for (wq = 0; wq <= 1 + np; wq++){ // wq = Anzahl weißer Damen
164     wnp1 = np - max(wq - 1, 0);
165     for (wr = 0; wr <= uwr + wnp1; wr++){ // wr = Anzahl weißer nicht-fixierter Türme
166         wnp2 = wnp1 - max(wr - uwr, 0);
167         for (wb = 0; wb <= 2 + wnp2; wb++){ // wb = Anzahl weißer Läufer
168             wnp3 = wnp2 - max(wb - 2, 0);
169             for (wn = 0; wn <= 2 + wnp3; wn++){ // wn = Anzahl weißer Springer
170                 wnp4 = wnp3 - max(wn - 2, 0);
171
172                 wproms = np - wnp4; // wproms = Anzahl weißer Promotionen
173
174                 wprod = fac[wk] * fac[wq] * fac[wr] * fac[wb] * fac[wn]; // Dient zur Berechnung eines Multinomialkoeff.
175
176                 for (wp = 0; wp <= wnp4; wp++){ // wp = Anzahl nicht-fixierter weißer Bauern
177                     wpx = np - wp - wproms; // wpx = Anzahl geschlagener weißer Bauern
178
179                     wpcs = wk + wq + wr + wb + wn; // wpcs = Anzahl weißer Pieces mit nichtfix König
180
181                     // Anzahl der schwarzen Figuren
182                     ubr = 2 - fbr; // Mindestanzahl nicht-fixierter schwarzer Türme
183                     bk = floor(ubr / 2); // Anzahl nicht-fixierter schwarzer Könige
184
185                     for (bq = 0; bq <= 1 + np; bq++){ // bq = Anzahl schwarzer Damen
186                         bnp1 = np - max(bq - 1, 0);
187                         for (br = 0; br <= ubr + bnp1; br++){ // br = Anzahl schwarzer nicht-fixierter Türme
188                             bnp2 = bnp1 - max(br - ubr, 0);
189                             for (bb = 0; bb <= 2 + bnp2; bb++){ // bb = Anzahl schwarzer Läufer
190                                 bnp3 = bnp2 - max(bb - 2, 0);
191                                 for (bn = 0; bn <= 2 + bnp3; bn++){ // bn = Anzahl schwarzer Springer
192                                     bnp4 = bnp3 - max(bn - 2, 0);
193
194                                     bproms = np - bnp4; // bproms = Anzahl schwarzer Promotionen
195
196                                     bprod = fac[bk] * fac[bq] * fac[br] * fac[bb] * fac[bn];
197
198                                     for (bp = 0; bp <= bnp4; bp++){ // bp = Anzahl nicht-fixierter schwarzer Bauern
199                                         bpx = np - bp - bproms; // bpx = Anzahl geschlagener schwarzer Bauern
200
201                                         bpcs = bk + bq + br + bb + bn; // bpcs = Anzahl schwarzer Pieces & nicht-fix. König
202
203                                         // Ist der weiße König fixiert ist fixwk = 1,
204                                         // sonst ist der nicht-fixierte weiße König, in wpcs enthalten. Für Schwarz analog
205                                         // caps = 32 - alle noch vorhandenen Figuren
206                                         caps = 32 - 2 * fixp - fixwk - fixbk - fwr - fbr - wp - bp - wpcs - bpcs;
207
208                                         maxuwp = bpx + caps - wproms; // Maximalanzahl weißer Bauern in freien Spalten
209                                         maxubp = bpx + caps - bproms; // Maximalanzahl schwarzer Bauern in freien Spalten
210                                         h = 0.0;
211                                         // Testet die Bedingung aus Lemma 4.7
212                                         if (maxuwp < 0 || maxubp < 0){
213                                             continue;
214                                         } else {
215                                             // Mindestanzahl blockierter Spalten
216                                             minopp = max(0, fixp + wp - maxuwp);
217                                             space = 64 - 4 * fixp - fixwk - fixbk - fwr - fbr - wp - bp;
218                                             h += pawns[fixp][wp][bp][minopp];
219                                             h *= fac[space] / (fac[space - wpcs - bpcs] * wprod * bprod);
220                                             summe += h;
221                                         }
222                                     }
223                                 }
224                             }
225                         }
226                     }
227                 }
228             }
229         }
230     }
231 }

```

```

224     }
225     }
226     }
227     }
228     }
229     }
230     }
231     }
232     }
233 }
234     return summe;
235 }
236
237 int main (){
238
239     //Initialisierung der Arrays
240     factorial();
241     choose2function();
242     moppsfuction();
243     pawnsfunction();
244     int fwr, fbr, fixp;
245     double res = 0.0;
246     double summe = 0.0;
247     double multFWR, multEP, multFBR;
248
249     // Alle möglichen Werte für fwr, fbr und fixp durchlaufen
250     for (fwr = 0; fwr <= 2; fwr++){
251
252         multFWR = (fwr == 1 ? 2 : 1); //multFWR
253
254         for (fbr = 0; fbr <= 2; fbr++){
255
256             multFBR = (fbr == 1 ? 2 : 1); //multFBR
257
258
259             for (fixp = 0; fixp <= 1; fixp++){
260
261                 multEP = (fixp == 1 ? 14 : 1); //multEP
262
263                 //Multiplikatoren heranmultiplizieren
264                 res = multEP * multFWR * multFBR * count(fwr, fbr, fixp);
265                 printf("fwr = %d\tfbr = %d\tfixp = %d\tgerundet: %e\texakt: %1.0f\n", fwr, fbr, fixp, res, res);
266                 summe += res;
267             }
268         }
269     }
270
271     // Anzahl wird mit 2 multipliziert um die Schachpositionen in denen Schwarz am Zug ist zu berücksichtigen
272     printf("Totale Summe: %1.0f \n", 2 * summe);
273     return 1;
274 }
275

```

meincode.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  extern double pow(double, double);
6
7  #define min(a,b) ((a) < (b) ? (a) : (b))
8  #define max(a,b) ((a) > (b) ? (a) : (b))
9
10 #define MNV 30          //Komponentenverschiebung im Feld mn
11 #define MBV 36          //Komponentenverschiebung im Feld mb
12
13
14 double fac[65];          // fac[n] = n!
15 double binom[65][11];    // binom[n][k] = Binomialkoeffizient von n über k
16 double choose2 [49][9][9]; // choose[n][j][k] = Multinomialkoeffizient  $n! / (j! * k! * (n - j - k)!)$ 
17 double mb [4][29][10][11]; // mb[kpos][s][wq][wr] = Obere Schranke aus Satz 5.3, wobei  $s + 36 = \text{space}$ 
18 double mn [34][11][11][11]; // mn[n][j][k][l] = Multinomialkoeffizient  $(n + 30)! / (j! * k! * l! * (n + 30 - j - k - l)!)$ 
19 double mopps_[9][33][9]; // Umsetzung von  $m(p,s,n)$  aus Satz 4.10
20 double pawns[2][9][9][9];
21
22 //Berechnet die Werte von fac[i]
23 void factorial() {
24     int i, j;
25     for (i=0; i<65; i++) {
26         fac[i] = 1.0;
27         for (j=2; j<=i; j++) {
28             fac[i] *= j;
29         }
30     }
31 }
32
33 // Berechnet die Werte von choose2[n][k1][k2]
34 void choose2function(){
35     int n, k1, k2;
36     for (n = 0; n <= 48; n++){
37         for (k1 = 0; k1 <= 8; k1++){
38             for (k2 = 0; k2 <= 8; k2++){
39                 if (n >= k1 + k2){
40                     choose2[n][k1][k2] = fac[n] / (fac[k1] * fac[k2] * fac[n - k1 - k2]);
41                 } else {
42                     choose2[n][k1][k2] = 0;
43                 }
44             }
45         }
46     }
47 }
48
49 //Berechnet die Werte von mopps[p][s][n]
50 double opps(int p, int s, int n) {
51     double res = 0.0;
52     if (p == 0 && s == 0) {
53         res = 1;
54     } else if (p == 0 || n == 0) {
55         res = 0;
56     } else {
57         res = mopps_[p][s][n - 1];
58         for (int i = 0; i <= (s < 4 ? s : 4); ++i) {
59             res += (5 - i) * mopps_[p - 1][s - i][n - 1];
60         }
61     }
62     return res;
63 }
64
65 //Weißt mopps[p][s][n], die in opps(p,s,n) berechneten Werte zu
66 void moppsfunction(){
67     double max = 0.0;
68     for (int p = 0; p <= 8; ++p) {
69         for (int s = 0; s <= 32; ++s) {
70             for (int n = 0; n <= 8; ++n) {
71                 mopps_[p][s][n] = opps(p, s, n);

```

```

72     }
73 }
74 }
75 }
76
77 // Berechnet die innere Summe in Satz 4.12
78 // dw = 1 <=> weißer Bauer in schwarzer fixierter Spalte
79 // db = 1 <=> schwarzer Bauer in weißer fixierter Spalte
80 // ds = ds1 + ds2 = Anzahl an eingeschlossener Felder in fixierten Spalten
81 double pawnsep (int dw, int db, int ds, int wp, int bp, int opp){
82     int opp_ = opp - dw - db;           // Mindestanzahl blockierter nicht-fixierter Spalten
83     int s;                             // Anzahl eingeschlossener Felder
84     double res = 0.0;
85     for (s = 0; s <= 4 * opp_; s++){
86         if(bp + dw >= opp && wp + db >= opp){ // Damit keine negative Komponente von choose2 aufgerufen wird
87             res += mopps_[opp_][s][6] * choose2[44 - opp - opp_ - ds - s][wp + db - opp][bp + dw - opp];
88         }
89     }
90     return res;
91 }
92
93
94 void pawnsfunction(){
95     int s, wp, bp, opp;
96     int ds1, ds2;
97     double res;
98
99     // Fall: Kein Enpassant -> fixp = 0
100    // Berechnet obere Schranke aus Satz 4.11
101    for (wp = 0; wp <= 8; wp++){
102        for (bp = 0; bp <= 8; bp++){
103            for (opp = 0; opp <= 8; opp++){
104                if (opp > min(wp, bp)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
105                    pawns[0][wp][bp][opp] = 0;
106                } else {
107                    res = 0.0;
108                    for (s = 0; s <= 4 * opp; s++){
109                        res += mopps_[opp][s][8] * choose2[48 - 2 * opp - s][wp - opp][bp - opp];
110                    }
111                    pawns[0][wp][bp][opp] = res;
112                }
113            }
114        }
115    }
116
117    // Fall: Enpassant -> fixp = 1
118    // Berechnet obere Schranke aus Satz 4.12
119    for (wp = 0; wp <= 8; wp++){
120        for (bp = 0; bp <= 8; bp++){
121            for (opp = 0; opp <= 8; opp++){
122                res = 0;
123                if (opp > min(wp + 1, bp + 1)){ // Es darf nicht mehr blockierte Spalten als Bauern einer Seite geben
124                    pawns[1][wp][bp][opp] = 0;
125                } else {
126                    res = pawnsep(0, 0, 0, wp, bp, opp);
127                    for (ds1 = 0; ds1 <= 2 && opp > 1; ds1++){
128                        for (ds2 = 0; ds2 <= 1; ds2++){
129                            res += pawnsep(1, 1, ds1 + ds2, wp, bp, opp);
130                        }
131                    }
132                    for (ds1 = 0; ds1 <= 2 && opp > 0; ds1++){
133                        res += pawnsep(1, 0, ds1, wp, bp, opp);
134                    }
135                    for (ds2 = 0; ds2 <= 1 && opp > 0; ds2++){
136                        res += pawnsep(0, 1, ds2, wp, bp, opp);
137                    }
138                    pawns[1][wp][bp][opp] = res;
139                }
140            }
141        }
142    }
143 }
144 }
145
146 //Beginn neuer Funktionen
147

```

```

148 // Berechnet n über k und speichert es in binom[n][k]
149 void binomial(){
150     int i, j;
151     for (i=0; i<65; i++) {
152         for (j=0; j<11; j++) {
153             if (i >= j) {
154                 binom[i][j] = fac[i] / (fac[j] * fac[i-j]);
155             } else {
156                 binom[i][j] = 0;
157             }
158         }
159     }
160 }
161
162 //Speichert den Wert  $(n + 30)! / (j! * k! * l! * (n + 30 - j - k - l)!)$  in mn[n][j][k][l]
163 void multinom(){
164     int spaces, f1, f2, f3; // Spaces wird hier um MNV verschoben
165     for (spaces = MNV; spaces <= 63; spaces++){
166         for (f1 = 0; f1 <= 10; f1++){
167             for (f2 = 0; f2 <= 10; f2++){
168                 for (f3 = 0; f3 <= 10; f3++){
169                     mn[spaces-MNV][f1][f2][f3] = fac[spaces] / (fac[f1] * fac[f2] * fac[f3] * fac[spaces - f1 - f2 - f3]);
170                 }
171             }
172         }
173     }
174 }
175
176 // Berechnet die obere Schranke aus Lemma 5.3 und speichert sie im Array mb
177 void multiplebinomrook (){
178     int kpos, space, wq, wr;
179     int qspaces;
180     for (kpos = 0; kpos <= 3; kpos++){
181         switch (kpos) {
182             case 0: qspaces = 1; break; // Ecke
183             case 1: qspaces = 2; break; // Letzte Reihe
184             case 2: qspaces = 0; break; // Bauernterritorium
185             case 3: qspaces = 2; break; // Fixierter König
186         }
187         for (space = MBV; space <= 64; space++){
188             for (wq = 0; wq <= 9; wq++){
189                 for (wr = 0; wr <= 10; wr++){
190                     mb[kpos][space-MBV][wq][wr] = binom[space - qspaces - 1][wq] * binom[space - wq - qspaces - 1][wr];
191                 }
192             }
193         }
194     }
195 }
196
197 // Ende neuer Funktionen
198
199 // Die count-Funktion wurde modifiziert, hier werden zur Übersichtlichkeit nur die modifizieren Stellen kommentiert
200
201 double count (int fwr, int fbr, int fixp){
202     double summe = 0.0;
203     double h = 0.0; // Hilfsvariable, damit der Code nicht zu weit nach rechts geht und gedruckt werden kann
204     int np = 8 - fixp;
205     int fixwk = (fwr != 0 ? 1 : 0);
206     int uwr, wk, wq, wnp1, wr, wb, wnp2, wnp3, wn, wnp4, wproms, wp, wpx;
207     int ubr, bk, bq, bnp1, br, bb, bnp2, bnp3, bn, bnp4, bproms, bp, bpx;
208     int fixbk = (fbr != 0 ? 1 : 0);
209     int wpcs, bpcs, caps, maxuwp, maxubp, minopp, space;
210     // wprod und bprod werden nicht mehr benötigt, da die Multinomialkoeffizienten mit dem mn-Array berechnet werden
211
212     // kpos und kmult kennen wir aus Kapitel 5, multFWK gibt die Anzahl an Möglichkeiten an den weißen König zu platzieren
213     // Die Platzierung des weißen Königs ist nach den Bauern, dem schwarzen König, den weißen Damen und weißen nicht-fixierten Türmen
214     int kpos, multFWK, kmult;
215     uwr = 2 - fwr;
216     wk = floor(uwr / 2);
217
218     // Hier wird geschaut, ob es keinen fixierten König gibt.
219     if (fbr == 0){
220
221         // Hier werden alle möglichen kpos eines nicht-fixierten Königs durchgegangen
222         for (kpos = 0; kpos <= 2; kpos++){
223             for (wq = 0; wq <= 1 + np; wq++){

```

```

224 wnp1 = np - max(wq - 1, 0);
225 for (wr = 0; wr <= uwr + wnp1; wr++){
226     wnp2 = wnp1 - max(wr - uwr, 0);
227     for (wb = 0; wb <= 2 + wnp2; wb++){
228         wnp3 = wnp2 - max(wb - 2, 0);
229         for (wn = 0; wn <= 2 + wnp3; wn++){
230             wnp4 = wnp3 - max(wn - 2, 0);
231             wproms = np - wnp4;
232             for (wp = 0; wp <= wnp4; wp++){
233                 wpx = np - wp - wproms;
234                 wpcs = wk + wq + wr + wb + wn;
235                 ubr = 2 - fbr;
236                 bk = floor(ubr / 2);
237                 for (bq = 0; bq <= 1 + np; bq++){
238                     bnp1 = np - max(bq - 1, 0);
239                     for (br = 0; br <= ubr + bnp1; br++){
240                         bnp2 = bnp1 - max(br - ubr, 0);
241                         for (bb = 0; bb <= 2 + bnp2; bb++){
242                             bnp3 = bnp2 - max(bb - 2, 0);
243                             for (bn = 0; bn <= 2 + bnp3; bn++){
244                                 bnp4 = bnp3 - max(bn - 2, 0);
245                                 bproms = np - bnp4;
246                                 for (bp = 0; bp <= bnp4; bp++){
247
248                                     bpx = np - bp - bproms;
249                                     bpcs = bk + bq + br + bb + bn;
250                                     caps = 32 - 2 * fixp - fixwk - fixbk - fwr - fbr - wp - bp - wpcs - bpcs;
251                                     maxuwp = bpx + caps - wproms;
252                                     maxubp = wpx + caps - bproms;
253                                     if (maxuwp < 0 || maxubp < 0){
254                                         continue;
255                                     } else {
256                                         minopp = max(0, fixp + wp - maxuwp);
257                                         // Wenn der weiße König fixiert ist, gibt es eine Möglichkeit den weißen König
258                                         // zu platzieren, ansonsten 63 - wr - wq - 4 * fixp - fbr - wp - bp
259                                         space = 64 - 4 * fixp - fwr - fbr - wp - bp - fixwk;
260                                         multFWK = (fwr > 0 ? 1 : space - 1 - wr - wq);
261                                         h = 0;
262                                         switch (kpos){
263                                             case 0: kmult = 4 - fwr; break;
264                                             case 1: kmult = 12 - 3 * fixwk - fwr; break;
265                                             case 2: kmult = 48 - 4 * fixp - wp - bp - 3 * fixwk - fwr; break;
266                                         }
267                                         // Der Multinomialkoeffizient von Tromp wird mithilfe des Ergebnisses der oberen
268                                         // Schranke aus Satz 5.3, multFWK und der Verteilung der restlichen Figuren
269                                         // als Multinomialkoeffizient ersetzt
270                                         h = pawns[fixp][wp][bp][minopp] * mb[kpos][space - MBV][wq][wr] * multFWK;
271                                         h *= kmult * mn[space - 2 - wq - wr - MNV + fixwk][wn][wb][bq];
272                                         h *= mn[space - 2 - wq - wn - wr - wb - bq - MNV + fixwk][br][bb][bn];
273                                         summe += h;
274                                     }
275                                 }
276                             }
277                         }
278                     }
279                 }
280             }
281         }
282     }
283 }
284
285 }
286 } else {
287     // Wenn der König fixiert ist, ist in jedem Fall kpos = 3 gegeben und damit auch kmult = 1
288     kpos = 3;
289     kmult = 1;
290     for (wq = 0; wq <= 1 + np; wq++){
291         wnp1 = np - max(wq - 1, 0);
292         for (wr = 0; wr <= uwr + wnp1; wr++){
293             wnp2 = wnp1 - max(wr - uwr, 0);
294             for (wb = 0; wb <= 2 + wnp2; wb++){
295                 wnp3 = wnp2 - max(wb - 2, 0);
296                 for (wn = 0; wn <= 2 + wnp3; wn++){
297                     wnp4 = wnp3 - max(wn - 2, 0);
298                     wproms = np - wnp4;
299                     for (wp = 0; wp <= wnp4; wp++){

```

```

300     wpx = np - wp - wproms;
301     wpcs = wk + wq + wr + wb + wn;
302
303     ubr = 2 - fbr;
304     bk = floor(ubr / 2);
305     np = 8 - fixp;
306     for (bq = 0; bq <= 1 + np; bq++){
307         bnp1 = np - max(bq - 1, 0);
308         for (br = 0; br <= ubr + bnp1; br++){
309             bnp2 = bnp1 - max(br - ubr, 0);
310             for (bb = 0; bb <= 2 + bnp2; bb++){
311                 bnp3 = bnp2 - max(bb - 2, 0);
312                 for (bn = 0; bn <= 2 + bnp3; bn++){
313                     bnp4 = bnp3 - max(bn - 2, 0);
314                     bproms = np - bnp4;
315                     for (bp = 0; bp <= bnp4; bp++){
316
317                         bpx = np - bp - bproms;
318                         bpcs = bk + bq + br + bb + bn;
319                         caps = 32 - 2 * fixp - fixwk - fixbk - fwr - fbr - wp - bp - wpcs - bpcs;
320                         maxuwp = bpx + caps - wproms;
321                         maxubp = wpx + caps - bproms;
322                         if (maxuwp < 0 || maxubp < 0){
323                             continue;
324                         } else {
325                             minopp = max(0, fixp + wp - maxuwp);
326                             space = 64 - 4 * fixp - fwr - fbr - wp - bp - fixwk;
327                             // Wenn der weiße König fixiert ist, gibt es eine Möglichkeit den weißen König zu
328                             // platzieren, ansonsten 63 - wr - wq - 4 * fixp - fbr - wp - bp
329                             multFWK = (fwr > 0 ? 1 : space - 1 - wr - wq);
330                             h = 0;
331
332                             // Der Multinomialkoeffizient von Tromp wird mithilfe des Ergebnisses der oberen
333                             // Schranke aus Satz 5.3, multFWK und der Verteilung der restlichen Figuren als
334                             // Multinomialkoeffizient ersetzt
335                             h = pawns[fixp][wp][bp][minopp] * mb[kpos][space - MBV][wq][wr] * multFWK;
336                             h *= kmult * mn[space - 2 - wq - wr - MNV + fixwk][wn][wb][bq];
337                             h *= mn[space - 2 - wq - wn - wr - wb - bq - MNV + fixwk][br][bb][bn];
338                             summe += h;
339                         }
340                     }
341                 }
342             }
343         }
344     }
345 }
346
347 }
348
349 }
350 }
351 return summe;
352 }
353
354 // In main gibt es wenig Unterschiede
355 // Es werden lediglich mehr Arrays initialisiert und am Ende wird noch ein Vergleich zu Tromps Code gezogen
356 int main () {
357     //Initialisierung der Arrays
358     factorial();
359     binomial();
360     choose2function();
361     multinom();
362     multiplebinomrook();
363     moppsfunction();
364     pawnsfunction();
365     int fwr, fbr, fixp;
366     double res = 0.0;
367     double summe = 0.0;
368     double multFWR, multEP, multFBR;
369
370     // Alle möglichen Werte für fwr, fbr und fixp durchlaufen
371     for (fwr = 0; fwr <= 2; fwr++){
372         multFWR = (fwr == 1 ? 2 : 1); //multFWR
373
374
375

```

```

376     for (fbr = 0; fbr <= 2; fbr++){
377
378         multFBR = (fbr == 1 ? 2 : 1); //multFBR
379
380
381         for (fixp = 0; fixp <= 1; fixp++){
382
383             multEP = (fixp == 1 ? 14 : 1); //multEP
384
385             //Multiplikatoren heranmultiplizieren
386             res = multEP * multFWR * multFBR * count(fwr, fbr, fixp);
387             printf("fwr = %d\tfbr = %d\tfixp = %d\tgerundet: %e\texakt: %1.0f\n", fwr, fbr, fixp, res, res);
388             summe += res;
389         }
390     }
391 }
392
393 // Ergebnis von Tromp
394 double haskellergebnis = 8726713169886222032347729969256422370854716254.0;
395
396 printf("Totale Summe: %1.0f \n", 2 * summe);
397 printf("Verbesserung um %1.1f Prozent\n", 100 * ((haskellergebnis - 2 * summe)/haskellergebnis));
398 return 1;
399 }

```